How To Scale Your Model

Exercise Solutions

Siddhartha Bhimireddy

July 13, 2026

Contents

Chapter 1

Intro to Rooflines

Q1: `int8` Matrix Multiplication

We want to perform $\texttt{int8}[B, D] \mathbin{@} \texttt{int8}[D, F] \longrightarrow \texttt{int8}[B, F]$.

Data movement Loading X requires BD bytes, since X contains BD parameters and each parameter occupies one byte. Similarly, loading Y requires DF bytes, and storing Z requires BF bytes. Therefore, the total communication is $BD + DF + BF$ bytes.

Operation count The matrix multiplication consists of BF dot products. Each dot product between two D-dimensional vectors requires D multiplications and $D-1$ additions. Therefore,

$$\begin{aligned}\text{total operations} &= BF\big(D + (D-1)\big)\\ &= BF(2D-1)\\ &= 2BDF - BF.\end{aligned}$$

Arithmetic intensity The arithmetic intensity is $I = \frac{BF(2D-1)}{BD+DF+BF}$.

If B is small, the DF term dominates the communication:

$$\begin{aligned}I &\approx \frac{BF(2D-1)}{DF}\\ &= B\left(\frac{2D-1}{D}\right).\end{aligned}$$

In the limit $D \to \infty$, $\frac{2D-1}{D} \longrightarrow 2$, so $I \approx 2B$.

Compute-bound threshold The accelerator intensity is

$$I_{\text{accelerator}} = \frac{3.94 \times 10^{14}}{8.2 \times 10^{11}}$$

$$= \frac{3.94 \times 10^3}{8.2}$$
$$\approx 480.$$

The computation becomes compute bound when

$$2B \geq 480,$$
$$B \geq 240.$$

Thus, $B = 240$ is the point after which the operation becomes compute bound.

Runtime bounds The mathematical and communication times are

$$t_{\text{math}} = \frac{BF(2D-1)}{3.94 \times 10^{14}},$$
$$t_{\text{comms}} = \frac{BD + DF + BF}{8.2 \times 10^{11}}.$$

The lower bound on the runtime is $\max(t_{\text{math}}, t_{\text{comms}})$, while the upper bound is $t_{\text{math}} + t_{\text{comms}}$.

Q2: Low-Precision Weights and High-Precision Activations

Store the weights in low precision while performing the operations and storing the activations in high precision: $\texttt{bf16}[B, D] \mathbin{@} \texttt{int8}[D, F] \longrightarrow \texttt{bf16}[B, F]$.

The communication cost is

$$T_{\text{comms}} = T_{\text{write}} + T_{\text{read}}$$
$$= 2BF + 2BD + DF.$$

The mathematical cost is

$$T_{\text{math}} = BF(2D-1)$$
$$\approx 2BDF.$$

The operation is compute bound when its arithmetic intensity exceeds the accelerator intensity. The operations take half as long as the `int8` operations, so

$$I_{\text{accelerator}} = \frac{1.97 \times 10^{14}}{8.2 \times 10^{11}}$$
$$\approx 240.$$

Therefore, the compute-bound condition is $\frac{2BDF}{2BF+2BD+DF} > 240$.

Assuming $B \ll D, F$,

$$\frac{2BDF}{2BF + 2BD + DF} \approx \frac{2BDF}{DF}$$

$$= 2B.$$

Thus,

$$2B \geq 240,$$
$$B \geq 120.$$

The operation becomes compute bound when $B \geq 120$.

Q3: Roofline Plots

Let $F = D = 4096$.

The number of bytes loaded is $2(4096)B + 4096^2$, and the number of bytes written is $2B(4096)$.

Therefore, the total memory movement is

$$M = 2(4096)B + 4096^2 + 2B(4096)$$
$$= 16384B + 16{,}777{,}216 \quad \text{bytes.}$$

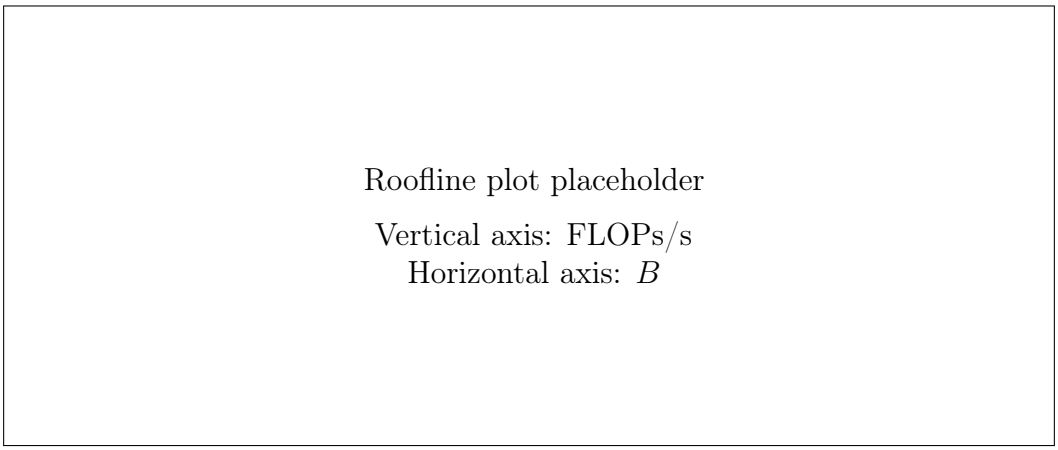

Figure 1.1: Roofline plot from the handwritten notes.

The number of operations is

$$BF(2D-1) = 4096B(8191)$$
$$= 33{,}550{,}336B.$$

The mathematical runtime is $\frac{33{,}550{,}336}{1.97\times10^{14}} = 1.70 \times 10^{-7}$ seconds.

The memory runtime is $\frac{16{,}777{,}216}{8.21\times10^{11}} = 2.04 \times 10^{-5}$ seconds.

Let $f =$ total FLOPs and $m =$ total memory bytes.

Then $\frac{f}{m} =$ arithmetic intensity.

The accelerator intensity is $\frac{\text{FLOPs/s}}{\text{bytes/s}}$.

The relationship between arithmetic intensity, bandwidth, and achieved performance is written as $\frac{f}{m} \cdot \frac{\text{FLOPs}}{\text{bytes}} \cdot \frac{\text{bytes}}{\text{s}} = \frac{\text{FLOPs}}{\text{s}}$.

Q4

Consider $\texttt{int8}[B, D] \circ_B \texttt{int8}[B, D, F] \longrightarrow \texttt{int8}[B, F]$.

Using a different matrix for each batch element does not change the number of FLOPs.

The communication and mathematical costs are

$$\begin{aligned} t_{\text{comms}} &= BD + BDF + BF, \\ t_{\text{math}} &= 2BDF. \end{aligned}$$

The arithmetic intensity is

$$\begin{aligned} \frac{t_{\text{math}}}{t_{\text{comms}}} &= \frac{2BDF}{BD + BDF + BF} \\ &= \frac{2BDF}{B(D + DF + F)} \\ &= \frac{2DF}{D + DF + F}. \end{aligned}$$

If $DF \gg D, F$, then the arithmetic intensity is approximately $I \approx 2$.

This is very low.

Q5

We have 1.979 teraFLOPs/s and 3.55 TB/s.

The accelerator intensity is

$$\begin{aligned} I_{\text{accelerator}} &= \frac{1.979 \times 10^{15}}{3.683 \times 10^{12}} \\ &\approx 537. \end{aligned}$$

The matrix-multiplication intensity is approximately B, so the matrix multiplication becomes compute bound when $B \geq 537$.

With integer sparsity, the operation threshold is

$$\frac{1.979 \times 10^{15}/2}{3.683 \times 10^{12}} = 268.5.$$

Therefore, the required batch size is $B \geq 269$.

Chapter 2

All About TPUs

Q1

Suppose we have a 200B parameter model stored in `bfloat16` and split across 32 TPUv4 chips.

The total parameter size is 200B parameters $\times$ 2 bytes $= 4.00 \times 10^{11}$ bytes. Since the parameters are evenly partitioned, $\frac{4.00\times10^{11}}{32}$ bytes are stored on each chip.

The TPUv4 HBM bandwidth is 1.2×10^{12} bytes/second. Therefore,

$$\begin{aligned} t_{\text{comms, per chip}} &= \frac{4.00 \times 10^{11} \text{ bytes}}{32 \cdot 1.2 \times 10^{12} \text{ bytes/s}} \\ &\approx 1.04 \times 10^{-2} \text{ s}. \end{aligned}$$

Since 1 ms $= 10^{-3}$ s, this is approximately 10 ms.

Q2

Full TPUv5e pod The total number of TPU hosts is computed as $\frac{\text{number of chips}}{\text{chips per host}} = \frac{16\times10\times16}{4\times2}$. Thus, $\frac{2560}{8} = 320$ hosts.

The total number of TPU cores is chips $\times 1 = 2560$. The total floating-point performance is $2560\times1.97\times10^{14} = 5.04\times10^{17}$ FLOPs/s. The total HBM bandwidth is $2560\times8.2\times10^{11} = 2.10 \times 10^{15}$ bytes/s.

Full TPUv6 pod The total number of hosts is $16 \times 20 \times 28 = 8960$ hosts.

Each chip contains two cores, giving $16 \times 20 \times 28 \times 2 = 17920$ cores.

The total compute is $(16 \times 20 \times 28) \times 4.59 \times 10^{14} = 4.11 \times 10^{18}$ FLOPs/s. The total HBM bandwidth is

$8960 \times 2.8 \times 10^{12} = 2.51 \times 10^{16}$ bytes/s.

Q3

Assume that the PCIe bandwidth is 1.6×10^{10} bytes/s. Let $A \in \texttt{bf16}^{D \times F}$, $X \in \texttt{bf16}^{B \times D}$, with $B \ll D$ and $F = 4D$. The accelerator intensity is

$$\begin{aligned} I_{\text{accelerator}} &= \frac{9.20 \times 10^{14} \text{ FLOPs/s}}{1.6 \times 10^{10} \text{ bytes/s}} \\ &= 57{,}500 \quad \frac{\text{FLOPs}}{\text{byte}}. \end{aligned}$$

Here, the communication time is dominated by PCIe.

The arithmetic intensity of the matrix multiplication is

$$I_{\text{matmul}} = \frac{BF(2D-1)}{2BD + 2DF + 2BF}.$$

Substituting $F = 4D$,

$$I_{\text{matmul}} = \frac{B(4D)(2D-1)}{2BD + 2D(4D) + 2B(4D)}.$$

Using $B \ll D$, this is approximately

$$\begin{aligned} I_{\text{matmul}} &\approx \frac{B(4D)(2D)}{2D(4D)} \\ &= B. \end{aligned}$$

Therefore, the matrix multiplication becomes compute bound when

$$B \geq 57{,}500.$$

Q4

Consider the matrix multiplication $\texttt{int8}[B, 4096] @ \texttt{int8}[4096, 16384]$, where B is unknown.

The mathematical runtime is

$$t_{\text{math}} = \frac{\text{total operations}}{3.94 \times 10^{14} \text{ FLOPs/s}}$$

$$= \frac{2B(4096)(16384)}{3.94 \times 10^{14}}$$
$$\approx 3.40 \times 10^{-7} B \quad \text{seconds.}$$

The total number of bytes transferred is

$$M = (4096)(16384) + B(4096) + B(16384).$$

Therefore, the communication time is

$$t_{\text{comms}} = \frac{(4096)(16384) + B(4096) + B(16384)}{8.2 \times 10^{11} \text{ bytes/s}}.$$

The minimum runtime is $t_{\text{min}} = \max(t_{\text{math}}, t_{\text{comms}})$. The accelerator intensity is

$$I_{\text{accelerator}} = \frac{3.94 \times 10^{14}}{8.2 \times 10^{11}}$$
$$\approx 480.$$

We want the arithmetic intensity to be at least 480:

$$\frac{2B(4096)(16384)}{(4096)(16384) + B(4096) + B(16384)} \geq 480.$$

Since $2(4096)(16384) \approx 1.34 \times 10^8$, we have

$$\frac{1.34 \times 10^8 B}{(4096)(16384) + 4096B + 16384B} \geq 480.$$

Multiplying through by the denominator gives

$$1.34 \times 10^8 B - 480(4096)B - 480(16384)B \geq 480(16384)(4096).$$

Approximating the terms,

$$B\left(1.34 \times 10^8 - 2 \times 10^6 - 8 \times 10^6\right) \geq 32 \times 10^9,$$

so

$$1.24 \times 10^8 B \geq 32 \times 10^9.$$

Thus,

$$\begin{aligned} B &\geq \frac{32 \times 10^9}{1.24 \times 10^8} \\ &\approx 258. \end{aligned}$$

Therefore, the matrix multiplication becomes compute bound at approximately $B \geq 258$. If the operands are loaded from VMEM, whose bandwidth is approximately 22 times the HBM bandwidth, the corresponding threshold is

$$\begin{aligned} B &\geq \frac{258}{22} \\ &\approx 11. \end{aligned}$$

Q5

Consider a TPUv5e arranged in a 4×4 slice.

We wish to transfer data from TPU$[0, 0, 3]$ to TPU$[3, 3, 3]$. No wrap-around links are used.

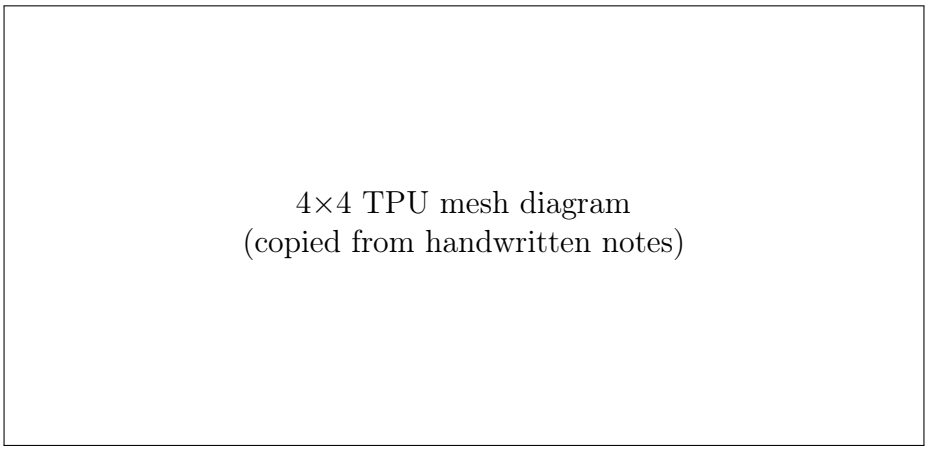

Figure 2.1: TPUv5e mesh used in Q5.

The ICI bandwidth is 4.5×10^{10} bytes/s. The total number of bytes transferred is

$$\begin{aligned} M &= 2 \times 128 \times 8 \times 8192 \\ &= 16\,777\,216 \text{ bytes}. \end{aligned}$$

The streaming time is

$$t_{\text{stream}} = \frac{16\,777\,216}{4.5 \times 10^{10}} \approx 3.72 \times 10^{-4} \text{ s.}$$

Since two links can be used simultaneously, $t_{\text{stream}} \approx \frac{3.72 \times 10^{-4}}{2} = 1.86 \times 10^{-4}$ s. The total latency is therefore $t_{\text{end-to-end}} = 6\ \mu\text{s} + 166\ \mu\text{s} \approx 172\ \mu\text{s}$.

Question 6

Matrix $A = \texttt{int8}\,[2^8 \times 1024,\ 128,\ 1024]$ is sharded evenly over a TPUv5e 4×4 slice.

We wish to compute $A[0, 0, 3] \cdot \texttt{bf16}[B, 128, 1024]$.

Strategy 1

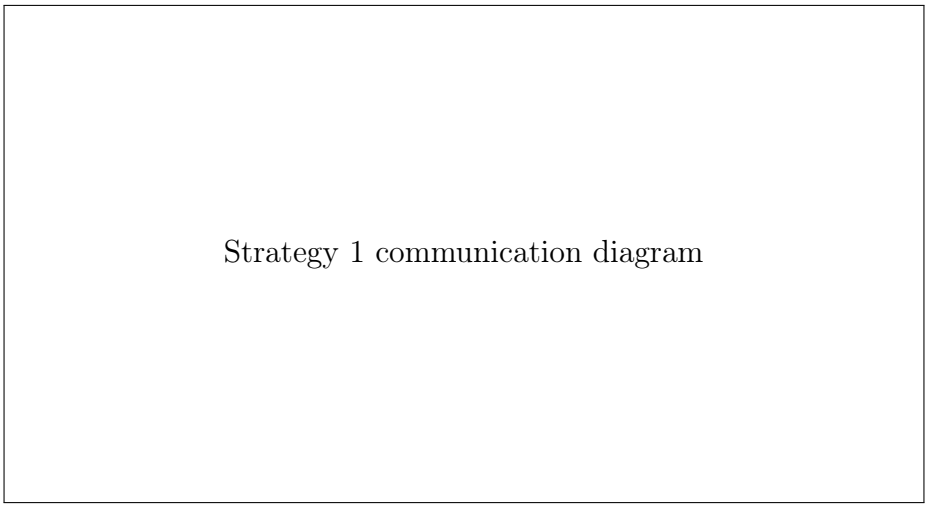

Figure 2.2: Strategy 1 from the handwritten notes.

Possible communication paths:

- HBM
- ICI
- PCIe
- DCN

Their roles are:

HBM: Transfer to VMEM.

ICI: Between neighboring TPU chips.

PCIe: Between the CPU host and its local TPU tray.

DCN: Between multiple CPU hosts.

The available DCN bandwidth is $3.125 \times 10^9 \times 8 = 2.5 \times 10^{10}$ bytes/s. The communication time over the DCN is

$$\begin{aligned} t_{\text{DCN}} &= \frac{16 \text{ GiB}}{2.5 \times 10^{10}} \\ &\approx 0.55 \text{ s}. \end{aligned}$$

The PCIe communication time is

$$\begin{aligned} t_{\text{PCIe}} &= \frac{16 \text{ GiB}}{1.6 \times 10^{10}} \\ &\approx 1.07 \text{ s}. \end{aligned}$$

The HBM communication is much faster and is therefore ignored in the critical path.

The mathematical runtime is

$$\begin{aligned} t_{\text{math}} &= \frac{2BDF}{3.94 \times 10^{14}} \\ &\approx 0.34 \text{ ms}. \end{aligned}$$

Hence this strategy is dominated by PCIe communication.

Strategy 2

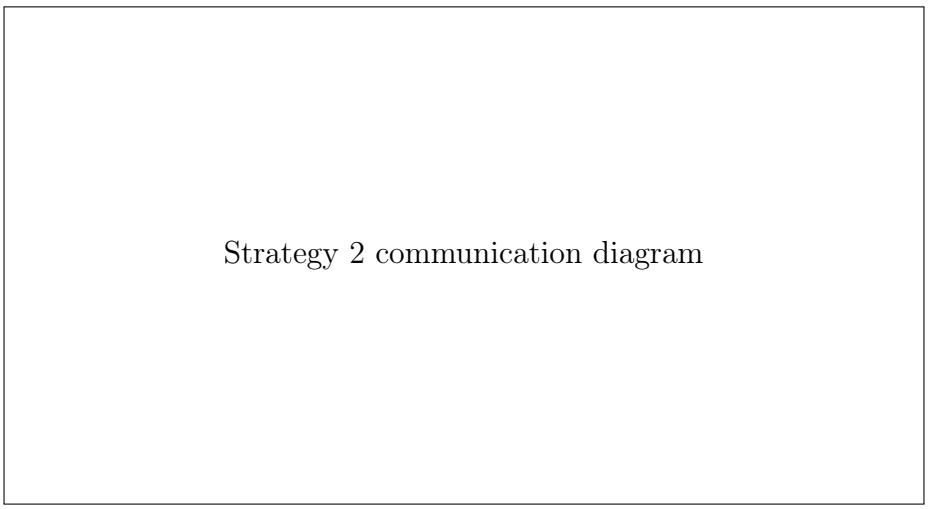

Figure 2.3: Strategy 2 from the handwritten notes.

Steps:

1. Load the data across every PCIe connection.
2. Hop to the destination using ICI.

Loading 16 GiB across all PCIe links gives approximately 67 ms. To reach the destination requires approximately six ICI hops.

Thus,

$$\begin{aligned} t_{\text{ICI}} &= \frac{16 \text{ GiB} \times 6}{16 \times 4.5 \times 10^{10}} \\ &\approx 0.147 \text{ s} \\ &= 147 \text{ ms}. \end{aligned}$$

Since 147 ms $>$ 67 ms, the runtime is dominated by ICI communication.

The runtime is bounded by $t_{\text{all}} \geq \max(t_{\text{comms}}, t_{\text{math}})$, and $t_{\text{all}} \leq t_{\text{comms}} + t_{\text{math}}$, assuming good overlap between communication and computation.

Therefore, the expected runtime is on the order of 147 ms.

Pop Quiz We have $8 \times 128 \times 4.175 \times 10^9 = 7.02 \times 10^{12}$ FLOPs/s.

Chapter 3

Sharded Matmuls

Pop Quiz 1

Consider an array $A \in \texttt{f32}[1024, 4096]$ with sharding $\mathcal{P}[A_{XY}, S]$ over the mesh $\{X : 8,\ Y : 2\}$. The local array on each device has shape

$$\left[\frac{1024}{8 \cdot 2}, 4096\right] = [64, 4096].$$

Therefore, each device stores

$$\begin{aligned} 64 \cdot 4096 \cdot 4 &= 1{,}048{,}576 \text{ bytes} \\ &\approx 1.04 \text{ MB}. \end{aligned}$$

The time required to load this from HBM is

$$\begin{aligned} t_{\text{HBM}} &= \frac{1.04 \times 10^6}{3.4 \times 10^{12}} \\ &\approx 3.0 \times 10^{-7} \text{ s} \\ &\approx 300 \text{ ns}. \end{aligned}$$

For reference,

$$\begin{aligned} 10^{9} &\sim \text{billion}, \\ 10^{-3} &\sim \text{millisecond}, \\ 10^{-6} &\sim \text{microsecond}, \\ 10^{-9} &\sim \text{nanosecond}. \end{aligned}$$

Pop Quiz 2

The TPUv5e bidirectional bandwidth is $W_{\text{ICI}} = 9.0 \times 10^{10}$ bytes/s. We want to perform $\text{AllGather}\left(\mathcal{P}[E_W, F]\right) \longrightarrow [E, F]$ over the mesh $\{X' : 8,\ W' : 4\}$, where $E = 2048$, $F = 8192$, and the array uses `bf16`.

The latency calculation is

$$\begin{aligned} t_{\text{latency}} &= 1\ \mu\text{s} \cdot \frac{\sum_i |X_i|}{2} \\ &= 1\ \mu\text{s} \cdot \frac{4}{2} \\ &= 2\ \mu\text{s}. \end{aligned}$$

The initial memory-transfer calculation is

$$\begin{aligned} t_{\text{transfer}} &= \frac{V}{W_{\text{ICI}} N_{\text{axes}}} \\ &= \frac{2048 \cdot 8192 \cdot 2}{(9.0 \times 10^{10}) \cdot 1} \\ &\approx 3.47 \times 10^{-4}\ \text{s} \\ &= 347\ \mu\text{s}. \end{aligned}$$

Since $347\ \mu\text{s} > 2\ \mu\text{s}$, this gives $t_{\text{total}} \approx 347\ \mu\text{s}$. However, the full bidirectional transfer cannot be used here. We must instead use the unidirectional bandwidth $W_{\text{ICI,uni}} = 4.5 \times 10^{10}$ bytes/s. The latency is now

$$\begin{aligned} t_{\text{latency}} &= 3\ \text{hops} \cdot 1\ \mu\text{s} \\ &= 3\ \mu\text{s}. \end{aligned}$$

The memory-transfer time is

$$\begin{aligned} t_{\text{transfer}} &= \frac{2048 \cdot 8192 \cdot 2 \cdot 3}{(4.5 \times 10^{10}) \cdot 4} \\ &\approx 5.59 \times 10^{-4}\ \text{s} \\ &= 559\ \mu\text{s}. \end{aligned}$$

This corresponds to transferring $\frac{V}{|X_i|}$ bytes per hop over three hops.

Pop Quiz 3

For an AllGather, the total time is $t_{\text{total}} = \max\left[T_{\min}\frac{\sum_i |X_i|}{2}, \frac{V}{W_{\text{ICI}} N_{\text{axes}}}\right]$. The first term is the latency term, where $T_{\min} = 1\ \mu\text{s}$, and the second term is the transfer term.

The TPUv5e bidirectional bandwidth is $W_{\text{ICI}} = 9.0 \times 10^{10}$ bytes/s.

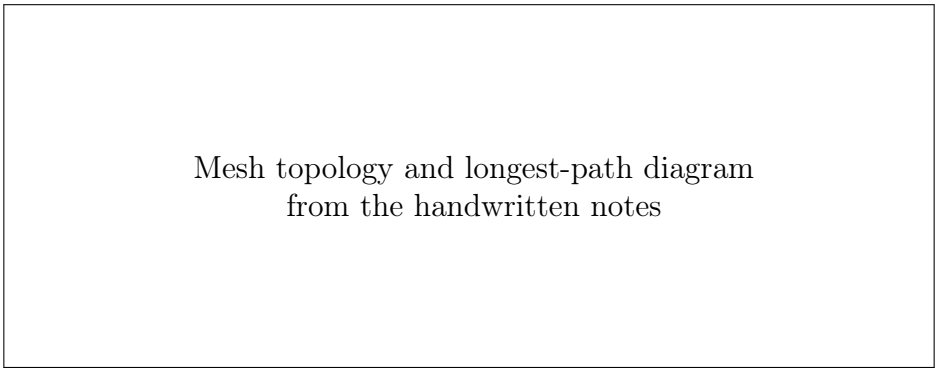

Figure 3.1: AllGather latency and transfer topology.

For the given mesh,

$$\begin{aligned} \frac{\sum_i |X_i|}{2} &= \frac{4}{2} \\ &= 2. \end{aligned}$$

For a `bf16` array of shape $[512, 8192]$, the transferred volume is $V = 2 \cdot 512 \cdot 8192$ bytes. Thus, $t_{\text{latency}} = 2$ μs. The memory-transfer time is

$$\begin{aligned} t_{\text{transfer}} &= \frac{2 \cdot 512 \cdot 8192}{(9.0 \times 10^{10}) \cdot 1} \\ &= 9.32 \times 10^{-5} \text{ s} \\ &\approx 91 \ \mu\text{s}. \end{aligned}$$

Q1

Suppose an array is sharded as $A[I_X, S, K, \ldots]$ over the mesh $\{X : 4,\ Y : 8,\ Z : 2\}$. We want the ratio between the total number of bytes occupied by A across all chips and the size of one copy of the array.

There are $4 \cdot 8 \cdot 2 = 64$ devices in the mesh.

Since A is sharded across Y, each partition contains $\frac{1}{8}$ of the full array. Therefore, the total storage across the mesh is

$$64 \cdot \frac{1}{8} = 8$$

copies of the array.

Thus, the ratio of total distributed storage to the size of one copy is $8 : 1$.

Q2

Consider a TPUv4 mesh $\{X : 4,\ Y : 4,\ Z : 4\}$, with $B = 1024,\quad D = 4096$, and `bf16` values.

We compare the following collectives:

$$\text{AllGather}_X([B_X, D]),\qquad \text{AllGather}_{XY}([B_{XY}, D]),\qquad \text{AllReduce}_Z([B, D]\{0_Z\}).$$

The TPUv4 unidirectional bandwidth is approximately 4.5×10^{10} bytes/s. Since the $4 \times 4 \times 4$ mesh has full bisection bandwidth, we use $W_{\text{ICI}} = 9.0 \times 10^{10}$ bytes/s.

AllGather over X

The latency is

$$\begin{aligned} t_{\text{latency}} &= 1\ \mu\text{s} \cdot \frac{\sum_i |X_i|}{2} \\ &= 1\ \mu\text{s} \cdot \frac{4}{2} \\ &= 2\ \mu\text{s}. \end{aligned}$$

The full array volume is $V = 2 \cdot 1024 \cdot 4096$ bytes. Since the transfer is distributed across the four links in the X dimension,

$$\begin{aligned} t_{\text{comms}} &= \frac{2 \cdot 1024 \cdot 4096}{4 \cdot 9.0 \times 10^{10}} \\ &= 2.33 \times 10^{-5}\ \text{s} \\ &\approx 23\ \mu\text{s}. \end{aligned}$$

AllGather over X and Y

The latency is

$$\begin{aligned} t_{\text{latency}} &= 1\ \mu\text{s} \cdot \frac{4+4}{2} \\ &= 4\ \mu\text{s}. \end{aligned}$$

For two mesh axes, $N_{\text{axes}} = 2$. Therefore,

$$t_{\text{comms}} = \frac{2 \cdot 1024 \cdot 4096}{(9.0 \times 10^{10}) \cdot 2}$$

$$\approx 4.6 \times 10^{-5} \text{ s}$$
$$\approx 46 \ \mu\text{s}.$$

Sharding over two axes introduces more replicas and therefore requires more communication.

AllReduce over Z

For the AllReduce over the Z axis, $|Z| = 4$. The latency is $t_{\text{latency}} = 2 \ \mu$s. An AllReduce consists of a ReduceScatter followed by an AllGather.

The volume per partition is $V = \frac{2 \cdot 1024 \cdot 4096}{4}$. The communication time for one phase is

$$t_{\text{phase}} = \frac{2 \cdot 1024 \cdot 4096}{4 \cdot 4 \cdot 9.0 \times 10^{10}}$$
$$\approx 5.82 \ \mu\text{s}.$$

Therefore,

$$t_{\text{AllReduce}} = t_{\text{ReduceScatter}} + t_{\text{AllGather}}$$
$$\approx 2(5.82 \ \mu\text{s})$$
$$= 11.64 \ \mu\text{s}.$$

Q3

Consider $\text{AllGather}_X([128])$ on a mesh $\{X : 4, \ Y : 4, \ Z : 4\}$, using `bf16` values and bandwidth $W_{\text{ICI}} = 9.0 \times 10^{10}$ bytes/s. The communication time is

$$t_{\text{comms}} = \frac{128 \cdot 2}{4 \cdot 9.0 \times 10^{10}}.$$

This is much less than 1 μs, so the collective is latency bound.

The latency is

$$t_{\text{latency}} = 1 \ \mu\text{s} \cdot \frac{4}{2}$$
$$= 2 \ \mu\text{s}.$$

Therefore, $t_{\text{total}} \approx 2 \ \mu$s.

Q4

We want to compute $X[B, D] \,@\, W[D_X, F] \longrightarrow Z[B, F]$. Assume TPUv4p, `bf16` values, and bidirectional ICI bandwidth.

We compare two strategies.

Strategy 1: AllGather the weights, then multiply

First perform $\text{AllGather}_X(W[D_X, F]) \longrightarrow W[D, F]$, and then compute $X[B, D] \,@\, W[D, F] \longrightarrow Z[B, F]$. The communicated volume is $V = 2DF$. The mathematical time is

$$t_{\text{math},1} = \frac{2BDF}{2.75 \times 10^{14}}.$$

The HBM communication time is

$$t_{\text{HBM},1} = \frac{BD + DF + BF}{1.2 \times 10^{12}}.$$

The ICI communication time is

$$t_{\text{ICI},1} = \frac{2DF}{9.0 \times 10^{10}}.$$

To compare the HBM and ICI terms, consider

$$\frac{BD + DF + BF}{1.2 \times 10^{12}} < \frac{2DF}{9.0 \times 10^{10}}.$$

Dividing by $2DF$,

$$\frac{B}{2F} + \frac{1}{2} + \frac{B}{2D} < \frac{1.2 \times 10^{12}}{9.0 \times 10^{10}}.$$

The right-hand side is approximately $\frac{120}{9} \approx 13$. Thus, in the regime considered here, the ICI communication dominates the HBM communication. We therefore approximate $t_{\text{comms},1} \approx \frac{2DF}{9.0 \times 10^{10}}$. Strategy 1 is communication bound when

$$\frac{2DF}{9.0 \times 10^{10}} > \frac{2BDF}{2.75 \times 10^{14}}.$$

Cancelling $2DF$,

$$\frac{1}{9.0 \times 10^{10}} > \frac{B}{2.75 \times 10^{14}},$$

so

$$\begin{aligned} B &< \frac{2.75 \times 10^{14}}{9.0 \times 10^{10}} \\ &\approx 3055. \end{aligned}$$

Therefore, Strategy 1 is communication bound when $B < 3055$.

Strategy 2: Local matmul, then AllReduce

Compute the local partial product $X[B, D] \text{ @ } W[D_X, F] \longrightarrow Z[B, F]\{0_X\}$, and then perform $\text{AllReduce}_X(Z[B, F]\{0_X\}) \longrightarrow Z[B, F]$. The local matmul is sharded across X, so

$$t_{\text{math},2} = \frac{2BDF}{X \cdot 2.75 \times 10^{14}}.$$

The AllReduce consists of a ReduceScatter and an AllGather. Its communication time is approximated by

$$t_{\text{comms},2} \approx \frac{4BF}{9.0 \times 10^{10}}.$$

Strategy 2 is communication bound when

$$\frac{2BDF}{X \cdot 2.75 \times 10^{14}} < \frac{4BF}{9.0 \times 10^{10}}.$$

Cancelling $2BF$,

$$\frac{D}{X \cdot 2.75 \times 10^{14}} < \frac{2}{9.0 \times 10^{10}}.$$

Equivalently,

$$\begin{aligned} \frac{D}{2X} &< \frac{2.75 \times 10^{14}}{9.0 \times 10^{10}} \\ &\approx 3055. \end{aligned}$$

Thus, $\frac{D}{6000} < X$.

Comparing the strategies

The total runtimes for the two strategies are

$$t_{\text{total},1} = \max\left(t_{\text{comms},1}, t_{\text{math},1}\right),$$
$$t_{\text{total},2} = \max\left(t_{\text{comms},2}, t_{\text{math},2}\right).$$

If $B < 3055$ and $\frac{D}{6000} < X$, then both strategies are communication bound. Their communication times are

$$t_{\text{comms},1} = \frac{2DF}{9.0 \times 10^{10}},$$
$$t_{\text{comms},2} = \frac{4BF}{9.0 \times 10^{10}}.$$

Strategy 1 is faster when

$$2DF < 4BF,$$

or $D < 2B$. Strategy 2 is faster when $D > 2B$. If $B < 3055$ and $\frac{D}{6000} \geq X$, then Strategy 1 is communication bound while Strategy 2 is compute bound.

For Strategy 1 to be faster, we would need

$$\frac{2DF}{9.0 \times 10^{10}} < \frac{2BDF}{X \cdot 2.75 \times 10^{14}}.$$

Cancelling $2DF$ gives

$$B > \frac{2.75 \times 10^{14}}{9.0 \times 10^{10}} X$$
$$\approx 3055X.$$

This contradicts $B < 3055$ for $X \geq 1$. Therefore, Strategy 2 is always better in this case.

If $B \geq 3055$ and $\frac{D}{6000} \geq X$, then both strategies are compute bound. Strategy 2 divides the mathematical work across X, so $t_{\text{math},2} = \frac{1}{X} t_{\text{math},1}$. Therefore, Strategy 2 is also better in this case.

Q5

We want to compute $A[I, J] \,@\, B[J, K] \longrightarrow C[I, K]$ on a TPUv4p $4 \times 4 \times 4$ mesh with the lowest possible latency.

The hardware values used in the notes are $W_{\text{ICI,uni}} = 4.5 \times 10^{10}$ bytes/s, $W_{\text{HBM}} = 1.2 \times 10^{12}$ bytes/s per chip,

and $P_{\text{bf16}} = 2.75 \times 10^{14}$ FLOPs/s per chip. The $4 \times 4 \times 4$ topology provides full bisection bandwidth.

Case 1: No sharding

With no sharding, the mathematical time is

$$t_{\text{math}} = \frac{2IJK}{2.75 \times 10^{14}}.$$

The HBM communication contains the two inputs and the output:

$$t_{\text{comms}} = \frac{2IJ + 2JK + 2IK}{1.2 \times 10^{12}}.$$

The total time is approximated by $t_{\text{total}} \approx \max(t_{\text{comms}}, t_{\text{math}})$. The operation is compute bound when

$$\begin{aligned} \frac{2IJK}{2IJ + 2JK + 2IK} &> \frac{2.75 \times 10^{14}}{1.2 \times 10^{12}} \\ &\approx 229. \end{aligned}$$

Case 2: Shard I across X

Compute $A[I_X, J] \,@\, B[J, K] \longrightarrow C[I_X, K]$, and perform an AllGather afterward if the full result is required.

Since $|X| = 4$, each chip performs one quarter of the computation:

$$t_{\text{math},X} = \frac{2IJK}{4 \cdot 2.75 \times 10^{14}}.$$

The communication is dominated by the AllGather of C:

$$t_{\text{comms},X} \approx \frac{2IK}{9.0 \times 10^{10}}.$$

Thus, $t_{\text{total},X} \approx \max\left(\frac{2IJK}{4 \cdot 2.75 \times 10^{14}}, \frac{2IK}{9.0 \times 10^{10}}\right)$. The sharded computation is compute bound when

$$\frac{2IJK}{4 \cdot 2.75 \times 10^{14}} > \frac{2IK}{9.0 \times 10^{10}}.$$

Cancelling $2IK$,

$$J > \frac{4 \cdot 2.75 \times 10^{14}}{9.0 \times 10^{10}}.$$

This gives a critical value of approximately $J \approx 12{,}222$. If the operation remains compute bound, sharding I across X reduces the mathematical latency.

Case 3: Shard I across X, Y, Z

Now compute $A[I_{XYZ}, J] \mathbin{@} B[J, K] \longrightarrow C[I_{XYZ}, K]$. Since $|XYZ| = 4 \cdot 4 \cdot 4 = 64$, the mathematical time is

$$\begin{aligned} t_{\text{math},XYZ} &= \frac{2IJK}{64 \cdot 2.75 \times 10^{14}} \\ &= \frac{IJK}{32 \cdot 2.75 \times 10^{14}}. \end{aligned}$$

The communication time is the cost of gathering the output shards:

$$t_{\text{comms},XYZ} \approx \frac{2IK}{2.7 \times 10^{10}}.$$

Comparing this communication time with the unsharded mathematical time gives a threshold of approximately $J < 71{,}000$. Thus, for $J < 71{,}000$, the fully sharded version $A[I_{XYZ}, J] \mathbin{@} B[J, K] \longrightarrow C[I_{XYZ}, K]$ is faster.

Other possibilities

One possibility is $A[I_X, J] \mathbin{@} B[J, K] \longrightarrow C[I_X, K]$, followed by an AllGather.

The I_X, I_{XY}, and I_{XYZ} variants have progressively lower mathematical time as the computation is distributed over more devices.

Since I_{XYZ} has the lowest mathematical time, it is the best choice once the operation is sufficiently compute bound.

Another possibility is to shard a contracting dimension: $A[I, J_X] \mathbin{@} B[J_X, K] \longrightarrow C[I, K]\{0_X\}$, followed by an AllReduce over X.

The AllReduce is approximately twice as expensive as an AllGather, so sharding the contracting dimension is generally worse than sharding a noncontracting dimension when both are otherwise possible.

Main takeaways

- The mathematical runtime while sharded is always lower than the mathematical runtime while unsharded. Therefore, if the operation remains compute bound after sharding, sharding is better.
- Becoming compute bound while sharded is more difficult, because the sharded communication time may become larger than the reduced mathematical time.
- In this example, this transition occurs around $J \approx 71{,}000$ when sharding over XYZ.
- Sharding a contracting dimension requires an AllReduce afterward. An AllReduce costs approximately twice as much as an AllGather while providing no additional FLOPs.
- The FLOP count depends on the number of ways the computation is sharded. Since $XYZ = 64$, sharding over XYZ provides the largest reduction in mathematical work per device.

Therefore, for this case, sharding as $A[I_{XYZ}, J] \,@\, B[J, K] \longrightarrow C[I_{XYZ}, K]$ and then performing an AllGather, or using an equivalent sharding over the noncontracting dimension, gives the lowest latency.

Q6

Assume `bf16`, so each parameter or activation occupies two bytes.

We consider a TPUv5e 4×4 mesh.

Part (a)

Consider $A[I_X, J_Y] \,@_J\, B[J_Y, K] \longrightarrow C[I_X, K]$. The local multiplication produces $C[I_X, K]\{0_Y\}$, so an AllReduce over Y is required: $\text{AllReduce}_Y\,(C[I_X, K]\{0_Y\}) \longrightarrow C[I_X, K]$. The mathematical time is

$$t_{\text{math}} = \frac{2IJK}{16 \cdot 1.97 \times 10^{14}}.$$

The AllReduce communication time is approximated by

$$t_{\text{comms}} = \frac{6IK}{16 \cdot 4.5 \times 10^{10}}.$$

The operation is compute bound when

$$\frac{2IJK}{16 \cdot 1.97 \times 10^{14}} > \frac{6IK}{16 \cdot 4.5 \times 10^{10}}.$$

Cancelling $2IK/16$,

$$\frac{J}{1.97 \times 10^{14}} > \frac{3}{4.5 \times 10^{10}}.$$

Therefore,

$$\begin{aligned} J &> \frac{3 \cdot 1.97 \times 10^{14}}{4.5 \times 10^{10}} \\ &\approx 13{,}100. \end{aligned}$$

Using the full six-transfer factor from the notes gives a threshold on the order of $J \approx 26{,}300$. Above this point, the operation is compute bound.

Part (b)

Consider $A[I_X, J] @ B[J_X, K_Y] \longrightarrow C[I_X, K_Y]$. First perform $\text{AllGather}_X(B[J_X, K_Y]) \longrightarrow B[J, K_Y]$, and then compute the matrix multiplication.

The AllGather communication cost is approximately one half the corresponding AllReduce cost.

The mathematical time is unchanged because $A[I_X, J] @ B[J, K_Y]$ is fully sharded and does not replicate computation.

Therefore, the critical value of J is of the same order as in the previous case.

Part (c)

Consider $A[I_X, J] @ B[J, K_Y] \longrightarrow C[I_X, K_Y]$. This requires the same mathematical work but no collective communication.

Part (d)

Consider a case where B must be gathered across X: $\text{AllGather}_X(B[J_X, K]) \longrightarrow B[J, K]$. The communication time is

$$t_{\text{comms}} = \left(\frac{JK}{4 \cdot 4}\right)\left(\frac{1}{4.5 \times 10^{10}}\right).$$

The mathematical time is

$$t_{\text{math}} = \frac{2IJK}{16 \cdot 1.97 \times 10^{14}}.$$

The computation dominates when

$$\frac{2IJK}{16 \cdot 1.97 \times 10^{14}} > \frac{JK}{16 \cdot 4.5 \times 10^{10}}.$$

Cancelling $JK/16$,

$$\frac{2I}{1.97 \times 10^{14}} > \frac{1}{4.5 \times 10^{10}}.$$

Therefore,

$$\begin{aligned} I &> \frac{1.97 \times 10^{14}}{2 \cdot 4.5 \times 10^{10}} \\ &\approx 2200. \end{aligned}$$

Using the transfer factors in the notes gives a practical threshold on the order of $I \approx 6500$.

Q7

The weight matrices occupy approximately 512 MB in total, so they must be sharded.

The activations occupy approximately 5 MB, so it is inexpensive to replicate or gather them.

Let X span the full 2×2 mesh. Shard the weights as $W_{\text{in}}[D, F_X]$, $W_{\text{out}}[F_X, D]$. First compute $\text{In}[B, D] \,@\, W_{\text{in}}[D, F_X] \longrightarrow \text{Temp}[B, F_X]$. Then compute $\text{Temp}[B, F_X] \,@\, W_{\text{out}}[F_X, D] \longrightarrow \text{Out}[B, D]\{0_X\}$. Finally, perform an AllReduce over X.

The AllReduce communication time is approximately

$$\begin{aligned} t_{\text{comms}} &= \frac{2 \cdot 2 \cdot 2 \cdot 128 \cdot 8192}{(4.5 \times 10^{10}) \cdot 4} \\ &\approx 4.55 \times 10^{-5} \text{ s}. \end{aligned}$$

This is a relatively costly AllReduce, although the collective is still small compared with the mathematical work.

We do not want to shard over D, because doing so would require an AllGather of the corresponding weight matrix.

The mathematical time for the two matrix multiplications is

$$t_{\text{math}} = \frac{2BDF + 2BDF}{1.97 \times 10^{14}}$$

$$\approx 6.97 \times 10^{-4} \text{ s}.$$

Since $t_{\text{math}} > t_{\text{comms}}$, the operation is compute bound.

Q8

The following are empirical measurements for AllGather, ReduceScatter, AllReduce, and AllToAll.

All measurements use `bf16` and one hop per collective.

The array shape is $[16384, 16384]$, with a $(4, 2)$ mesh over axes (X, Y).

AllGather

The operation is $[D_X, F] \longrightarrow [D, F]$, and the measured runtime is $14.2 \text{ ms} \pm 220\ \mu\text{s}$.

ReduceScatter

The operation is $[D, F]\{0_X\} \longrightarrow [D_X, F]$, and the measured runtime is approximately 11.7 ms.

AllReduce

The operation is $[D, F]\{0_X\} \longrightarrow [D, F]$, and the measured runtime is approximately 19.4 ms.

AllToAll

The operation is $[D_X, F] \longrightarrow [D, F_X]$, and the measured runtime is $4.75 \text{ ms} \pm 197\ \mu\text{s}$. It is surprising that ReduceScatter and AllGather have different runtimes, since they might be expected to have comparable communication costs.

A possible explanation is that ReduceScatter can overlap some compute with communication.

The measurements also suggest that $t_{\text{AllReduce}} \approx 2t_{\text{ReduceScatter}}$, while AllToAll has substantially lower runtime, up to fixed overhead and latency.

Q9

Consider $A[N, M] \,@\, B[M, K]$. We compare two explicit distributed algorithms.

Algorithm 1

On each device, compute outer products between rows or vectors of A and B.

For example, $C[N, K] \mathrel{+}= a_{:,m} b_{m,:}$. Each device computes a subset of the outer products. Once the local partial results have been accumulated, perform an AllReduce to fully replicate the output.

Each device must have access to the corresponding rows or vectors of B required for its local computation.

Algorithm 2

If the relevant matrix dimension is not replicated, the final collective can instead become a ReduceScatter.

Schematically, $C[N, K]\{0_X\} \longrightarrow C[N, K_X]$ or an equivalent output sharding.

AllGather and AllReduce have comparable communication costs in this model. Therefore, the communication volume is proportional to the activation being transferred.

The two relevant volumes are on the order of $2NM$ and $2MK$. Algorithm 2 is better when

$$2MK > 2NM,$$

which simplifies to $K > N$.

Q10: AllToAll

Consider an array with sharding $A[I_X, J]$. Let the relevant matrix size be $N \times N$, distributed across D devices.

1. AllGather communication

For an AllGather, each device initially holds $\frac{N^2}{D}$ scalars.

The data must be transferred through the ring over $D-1$ stages. Thus, the scalar-transfer count is proportional to $\frac{N^2}{D}(D-1)$. With bidirectional links, the same total information is transferred, but the traffic can travel in both directions around the ring.

2. Mini-chunk interpretation

Divide the distributed array into mini-chunks. The number of scalars in one mini-chunk is $\frac{N^2}{D^2}$. During an AllGather, the mini-chunks move through progressively more links until every device has received every chunk.

The total number of mini-chunk transfers around the ring is

$$\sum_{n=1}^{D-1} n = \frac{(D-1)D}{2}.$$

3. Comparing AllGather and AllToAll

For AllGather, the total scalar-transfer count is

$$V_{\text{AllGather}} = \frac{N^2}{D^2} D(D-1).$$

For AllToAll, it is

$$V_{\text{AllToAll}} = \frac{N^2}{D^2} \frac{D(D-1)}{2}.$$

The factor of $1/2$ arises because AllGather sends the entire shard through $D-1$ stages, whereas AllToAll sends progressively smaller fractions.

The sequence of fractions is $\frac{D-1}{D}, \frac{D-2}{D}, \ldots, \frac{1}{D}$. Using $\sum_{n=1}^{D-1} n = \frac{D(D-1)}{2}$, the total is smaller by a factor of approximately two.

4. Effect of bidirectional links

For AllGather, two links can operate simultaneously, which gives approximately twice the speed of a unidirectional implementation.

The total number of scalars transferred is unchanged, but the traffic is split between the two directions around the ring. Therefore, the runtime is approximately halved.

5. Bidirectional AllToAll

In bidirectional AllToAll, approximately half of the chunks travel to the left and half travel to the right.

For one direction, the number of mini-chunk transfers is

$$\sum_{i=1}^{D/2-1} i = \frac{\left(\frac{D}{2}-1\right)\left(\frac{D}{2}\right)}{2}.$$

For the odd-D expression used in the notes,

$$\sum_{i=1}^{(D-1)/2} i = \frac{\left(\frac{D-1}{2}\right)\left(\frac{D+1}{2}\right)}{2} = \frac{D^2-1}{8}.$$

Therefore, the bidirectional AllToAll scalar-transfer count per link is

$$V_{\text{AllToAll,bidir}} = \frac{N^2}{D^2}\frac{D^2-1}{8}.$$

By comparison, the AllGather transfer count is

$$V_{\text{AllGather}} = \frac{N^2}{D^2}\frac{D(D-1)}{2}.$$

Thus, bidirectional AllToAll transfers approximately four times fewer bytes through each link.

6. Final comparison

Unidirectional AllToAll is approximately twice as fast as unidirectional AllGather because AllToAll does not replicate every chunk on every device.

Bidirectional communication provides another factor of approximately two.

Therefore, $t_{\text{AllToAll,bidir}} \approx \frac{1}{4} t_{\text{AllGather,bidir}}$. Equivalently, bidirectional AllToAll is approximately four times faster than bidirectional AllGather.

The key reason is that AllToAll transfers only the chunks required by each destination, rather than fully replicating all shards.

Chapter 4

Transformers

Q1

Assume $K = N$. Let $D = 4096$, $F = 4D$, $V = 32{,}000$, $L = 64$. The total number of parameters is

$$P = L\left(3DF + 2D(N+K)H\right) + DV.$$

Using $K = N$ and $NH = D$,

$$\begin{aligned} P &= L\left(3DF + 4DNH\right) + DV \\ &= L\left(3DF + 4D^2\right) + DV \\ &\approx 17.3 \times 10^9. \end{aligned}$$

Thus, the model contains approximately 17.3 billion parameters. The fraction of the model occupied by the attention parameters is

$$\begin{aligned} \frac{L(4D^2)}{P} &= \frac{64(4D^2)}{17.3 \times 10^9} \\ &\approx 25\%. \end{aligned}$$

The KV cache contains $2LKH$ `bf16` values per token. Since each value occupies two bytes, the cache size per token is

$$\begin{aligned} M_{\text{KV/token}} &= 2\left(2LKH\right) \text{ bytes} \\ &\approx 524 \text{ KB}. \end{aligned}$$

Q2

To compute $A[B, D] \,@\, W[D, F]$, we perform $2BDF$ FLOPs.

The matrices are sharded over X and Y, but not over Z. Therefore, there are four replicas of the matrix, and the total amount of computation across the mesh is

$4(2BDF) = 8BDF$ FLOPs.

For the mesh $\{X : 4,\ Y : 8,\ Z : 4\}$, there are $4 \cdot 8 \cdot 4 = 128$ devices. Therefore, each TPU performs

$$\frac{8BDF}{128} = \frac{BDF}{16}$$

FLOPs.

Q3

Consider $A[I, J, K, L] * B[I, J, M, N, O] \longrightarrow C[K, L, M, N, O]$. We contract over I and J, with no batch dimensions.

Therefore, the operation requires $2IJKLMNO$ FLOPs.

Q4

We calculate the arithmetic intensity of self-attention in terms of B, T, S, N, K, and H.

Assuming no KV cache, self-attention from a fresh computation requires $4BT^2NH$ FLOPs.

The Q, K, and V activations require approximately $BTNH + 2BSKH$ values to be loaded. If $N = K$ and $S = T$, this becomes $3BTNH$. Therefore, the arithmetic intensity is

$$\begin{aligned} I_{\text{attention}} &= \frac{4BT^2NH}{3BTNH} \\ &= \frac{4T}{3}. \end{aligned}$$

Thus, attention is compute bound approximately when $T > 225$. When $T < 225$, communication dominates the attention operation.

Feedforward Layer

A feedforward layer performs three matrix multiplications and requires $6BTDF$ FLOPs.

Loading the weights requires approximately $3DF$ bytes, giving the arithmetic intensity

$$I_{\text{FFW}} = \frac{6BTDF}{3DF} = 2BT.$$

Assuming $B = 8$, the feedforward layer becomes compute bound approximately when $T > 28$. The communication volumes are

$$M_{\text{attention}} = 3BSKH,$$
$$M_{\text{FFW}} = 3DF + 2BTD + 2BTF.$$

Assuming $F = 4D$, $D \gg TB$, $KH = D$, the feedforward communication clearly dominates.

Therefore, when $T < 225$, the feedforward layer takes longer.

Compute-Bound Regime

When $T > 225$, attention is compute bound and requires $4BT^2NH$ FLOPs.

Using $NH = D$, the attention FLOPs become $4BT^2D$. The feedforward layer requires $6BTDF$. Using $F = 4D$, this becomes $24BTD^2$. The ratio between the attention FLOPs and the feedforward FLOPs is

$$\frac{4BT^2D}{24BTD^2} = \frac{T}{6D}.$$

Attention takes more time when

$$\frac{T}{6D} > 1,$$

or equivalently, $T > 6D$. For the value of D used here, this occurs at approximately $T > 48{,}300$.

Q5

The FLOP count for self-attention is $F_{\text{attention}} = 4BT^2NH$. The FLOP count for the attention projections is $F_{\text{projection}} = 4BTDH(N + K)$. To find when the two costs are equal, set

$$4BT^2NH = 4BTDH(N + K).$$

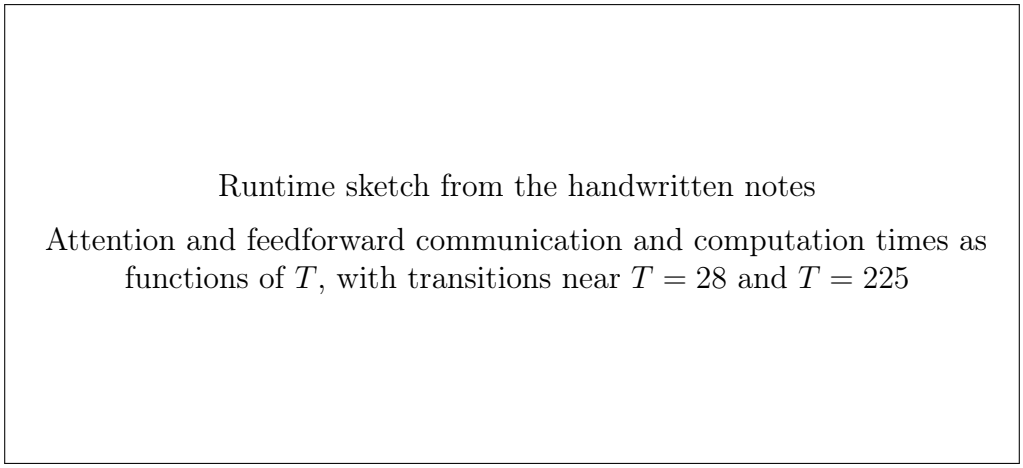

Figure 4.1: Communication-bound and compute-bound regimes.

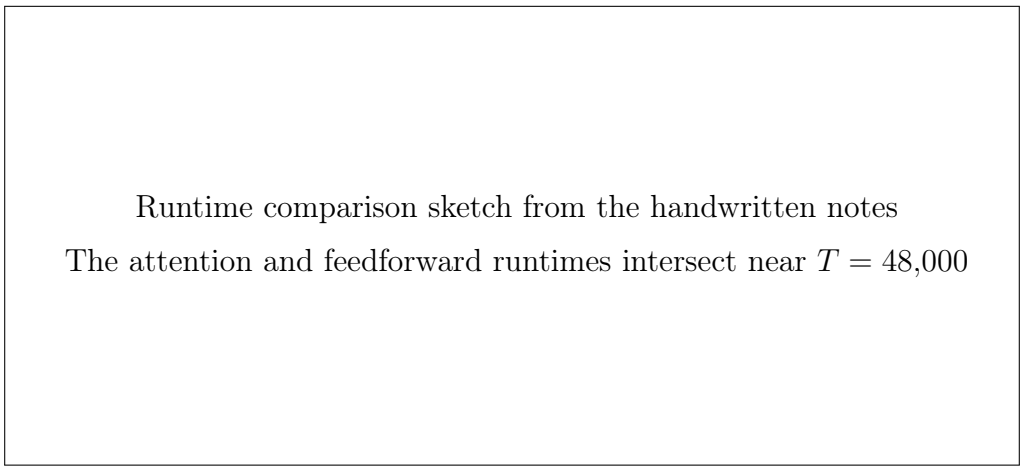

Figure 4.2: Point at which attention becomes slower than the feedforward layer.

Cancelling the common factors gives

$$TN = D(N + K).$$

Assuming $K = N$, we obtain

$$\begin{aligned} TN &= 2DN, \\ T &= 2D. \end{aligned}$$

Thus, the attention and projection FLOP counts are equal when $\boxed{T = 2D}$.

Q6

Suppose we save only the outputs of the seven matrix multiplications: Q, K, V, O, and the three feedforward matrix multiplications.

We want to determine how many additional FLOPs are required to resynthesize the remaining activations during the backward pass.

The operations that must be recomputed include:

- dot-product attention,
- softmax,
- the multiplication by the values,
- residual connections,
- normalization,
- GeLU,
- the elementwise multiplication in the feedforward layer,
- and the remaining normalization.

Layer normalization requires $O(BTD)$ operations, and the residual connections also require $O(BTD)$ operations. The recomputation cost is therefore dominated by the attention matrix multiplications.

The first attention matrix multiplication has cost

$$[B, T, K, G, H] \mathbin{@} [B, S, K, H] \longrightarrow 2BTSKGH \quad \text{FLOPs}.$$

Assuming $T = S$, this becomes $2BT^2KGH$. The second attention matrix multiplication has the same cost:

$$\begin{aligned}[B, T, S, K, G] \mathbin{@} [B, S, K, H] &\longrightarrow 2BTSKGH \\ &= 2BT^2KGH.\end{aligned}$$

Thus, the total recomputation cost is

$$\begin{aligned}F_{\text{remat}} &= 2BT^2KGH + 2BT^2KGH \\ &= \boxed{4BT^2KGH}.\end{aligned}$$

Q7

We estimate the FP8 training FLOPs without structured sparsity.

A rough estimate for the total training FLOPs is $6 \times$ tokens $\times$ activated parameters. The number of activated parameters is 37×10^9, and the number of training tokens is 14.8×10^{12}. Therefore, the a priori estimate is

$$\begin{aligned} F_{\text{realized}} &= 6\left(37 \times 10^9\right)\left(14.8 \times 10^{12}\right) \\ &= 3.28 \times 10^{24} \quad \text{training FLOPs.} \end{aligned}$$

The actual hardware usage was 2.79 million H800-hours. An H800 achieves approximately 3026 FP8 TFLOPs/s with structured sparsity. Without structured sparsity, this is approximately 1513 FP8 TFLOPs/s. Thus, the total theoretical hardware FLOPs are

$$\begin{aligned} F_{\text{theoretical}} &= \left(2.79 \times 10^6\right)(60)(60)\left(1513 \times 10^{12}\right) \\ &= 1.520 \times 10^{25}. \end{aligned}$$

The hardware utilization is therefore

$$\begin{aligned} \text{utilization} &= \frac{F_{\text{realized}}}{F_{\text{theoretical}}} \\ &= \frac{3.28 \times 10^{24}}{1.520 \times 10^{25}} \\ &\approx \boxed{21.6\%}. \end{aligned}$$

Q8

To overlap the operations, we want to load all E experts into memory.

Assume that the entire MoE layer is placed on one TPUv5e and that the expert weights are loaded directly from HBM.

Each token is routed to K experts. Each expert feedforward network performs three matrix multiplications, so the mathematical work is

$$\begin{aligned} F_{\text{math}} &= K(3)(2BTDF) \\ &= 6KBTDF. \end{aligned}$$

The mathematical runtime is therefore $t_{\text{math}} = \frac{6KBTDF}{\text{accelerator FLOPs/s}}$. The communication volume is $BTF + BTD + 3DFE$, so $t_{\text{comms}} = \frac{BTF+BTD+3DFE}{\text{accelerator bandwidth}}$. The arithmetic intensity of the MoE layer is

$$I_{\text{MoE}} = \frac{K(6BTDF)}{BTF + BTD + 3DFE}.$$

If B is small, the weight-loading term dominates the denominator. Therefore,

$$\begin{aligned} I_{\text{MoE}} &\approx \frac{K(6BTDF)}{3DFE} \\ &= \frac{2KBT}{E}. \end{aligned}$$

For TPUv5e, the accelerator intensity is

$$\begin{aligned} I_{\text{accelerator}} &= \frac{1.97 \times 10^{14}}{8.2 \times 10^{11}} \\ &\approx 240. \end{aligned}$$

The MoE layer is compute bound when

$$\frac{2KBT}{E} > 240.$$

Equivalently,

$$BT > \frac{120E}{K}.$$

For DeepSeek, using $K = 8$ and $E = 256$, we require

$$\begin{aligned} BT &> \frac{120(256)}{8} \\ &= \boxed{3840}. \end{aligned}$$

Chapter 5

Training

Q1: LLaMA-2 13B Parameter Calculation

The parameter count is $P = L\left(3DF + 2D(N+K)H + O(D)\right) + DV$, where the first term contains the transformer layers and DV contains the unembedding parameters.

Using $D = 5120$, $F = 13824$, $N + K = 80$, $H = 128$, $L = 40$, $V = 32000$, we obtain

$$\begin{aligned} P &= \left[3(5120)(13824) + 2(5120)(80)(128) + O(5120)\right](40) + (5120)(32000) \\ &= 12{,}852{,}019{,}200 \\ &\approx 13 \times 10^9. \end{aligned}$$

Thus, the model contains approximately $\boxed{\text{13 billion parameters}}$.

Q2

Assume $BS = 16$ million tokens and use Adam.

Each token contains an activation of size [5120]. Since the activations use `bfloat16`, each token requires

$$5120 \cdot 2 = 10240 \text{ bytes.}$$

We checkpoint after each feedforward matrix multiplication. The activation memory per layer is

$$\begin{aligned} M_{\text{act,layer}} &= 2\left(2(BS)F + (BS)D\right) \\ &= 4(16 \times 10^6)(13824) + 2(16 \times 10^6)(5120) \end{aligned}$$

$$\approx 1.05 \times 10^{12} \text{ bytes.}$$

Thus, each layer requires approximately 1.05 TB of activation memory.

For $L = 40$ layers,

$$\begin{aligned} M_{\text{activations}} &= 40(1.05 \text{ TB}) \\ &\approx 42 \text{ TB.} \end{aligned}$$

The model contains approximately 13 billion parameters. With `bfloat16` weights,

$$\begin{aligned} M_{\text{parameters}} &= 13 \times 10^9 \cdot 2 \\ &= 26 \times 10^9 \text{ bytes} \\ &= 26 \text{ GB.} \end{aligned}$$

For Adam, we store eight bytes of optimizer state per parameter:

$$\begin{aligned} M_{\text{optimizer}} &= 8(13 \times 10^9) \\ &= 104 \text{ GB.} \end{aligned}$$

Therefore, the total memory requirement is approximately

$$\begin{aligned} M_{\text{total}} &= 42 \text{ TB} + 26 \text{ GB} + 104 \text{ GB} \\ &= \boxed{42.130 \text{ TB}}. \end{aligned}$$

Q3

Assume:

$$\text{context length} = 32\text{K tokens},$$

$B = 3 \times 10^6$ total tokens per batch,

and a TPUv5p slice of size $16 \times 16 \times 16 = 4096$ chips. The weights use `bfloat16`, while the optimizer state uses `float32`.

Part 1: Replicated Model

The TPUv5p has approximately 96 GB of HBM per chip.

The parameters and optimizer state require

$$\begin{aligned} M_{\text{parameters+optimizer}} &= 26 \text{ GB} + 104 \text{ GB} \\ &= 130 \text{ GB}. \end{aligned}$$

Since 130 GB $>$ 96 GB, the parameters and optimizer state cannot be replicated on each chip:

$$\boxed{\text{No}}.$$

Part 2: Pure FSDP

With FSDP, the weights and optimizer state are distributed across the data-parallel shards and gathered or reduced as needed.

The parameter and optimizer-state memory per chip is $\frac{130 \text{ GB}}{4096}$. The total activation memory is approximately 42 TB. Sharding the activations over all 4096 devices gives

$$\begin{aligned} M_{\text{activations/chip}} &= \frac{42 \text{ TB}}{4096} \\ &\approx 10.25 \text{ GB}. \end{aligned}$$

Therefore, pure FSDP fits comfortably in memory.

However, the per-shard batch size is

$$\frac{3 \times 10^6}{4096} \approx 732.$$

Since $732 < 840$, the pure-FSDP configuration is not compute bound.

Part 3: Combining FSDP and Tensor Parallelism

Let $N = 4096$ be the total number of chips. Let X be the FSDP dimension and Y be the tensor-parallel dimension, so that $XY = N$. The optimal value of X is

$$\begin{aligned} X_{\text{opt}} &= \sqrt{\frac{B}{F}\frac{M_x}{M_y}N} \\ &= \sqrt{\frac{3 \times 10^6}{13824}(4096)}\sqrt{\frac{M_x}{M_y}} \end{aligned}$$

$$\approx 942\sqrt{\frac{M_x}{M_y}}.$$

If

$M_x = 1, \qquad M_y = 2$, then

$$\begin{aligned} X_{\text{opt}} &\approx 942\sqrt{\frac{1}{2}} \\ &\approx 666. \end{aligned}$$

The nearest convenient mesh size is approximately $X \approx 512$. If instead

$M_x = 2, \qquad M_y = 1$, then

$$\begin{aligned} X_{\text{opt}} &\approx 942\sqrt{2} \\ &\approx 1332. \end{aligned}$$

The nearest convenient mesh size is approximately $X \approx 1024$.

Compute-bound condition For $X = 512$, in the case $M_x = 1, \qquad M_y = 2$, the compute-bound condition is $\frac{B}{N} > \frac{\alpha^2}{M_x M_y F}$. The same batch-size condition applies regardless of whether $(M_x, M_y) = (1, 2)$ or $(M_x, M_y) = (2, 1)$, because in either case $M_x M_y = 2$.

Communication balance The communication cost is proportional to $t_{\text{comms}} \propto \max\left(\frac{F}{YM_x}, \frac{B}{XM_y}\right)$. At the optimum, the two terms are equal.

Suppose X^* and Y^* are optimal when $M_x = 2, \qquad M_y = 1$. Then $\frac{F}{2Y^*} = \frac{B}{X^*}$. For the reversed assignment, $M_x = 1, \qquad M_y = 2$, let

$Y' = 2Y^*, \qquad X' = \frac{X^*}{2}$. Then

$$\begin{aligned} \frac{F}{Y'} &= \frac{F}{2Y^*}, \\ \frac{B}{2X'} &= \frac{B}{X^*}. \end{aligned}$$

Therefore, $\frac{F}{Y'} = \frac{B}{2X'}$. Thus, the communication time at the respective optima is independent of whether $M_x = 1$ or $M_y = 1$. The optimal mesh dimensions change by the corresponding factors of M_x and M_y, but the resulting communication time is the same.

Chosen configuration Take $X \approx 512$, giving 512-way FSDP and 8-way tensor parallelism, since $512 \cdot 8 = 4096$. The parameter and optimizer-state memory per chip is

$$\frac{130 \text{ GB}}{512 \cdot 8} = \frac{130 \text{ GB}}{4096}.$$

The activations are also sharded over X and Y, so the activation memory per chip remains

$$\frac{42 \text{ TB}}{4096} \approx 10.25 \text{ GB}.$$

Checking whether the configuration is compute bound Using $\alpha = 2550$, $F = 13824$, $M_x M_y = 2$, the compute-bound condition becomes

$$\begin{aligned} \frac{B}{N} &> \frac{2550^2}{(13824)(2)} \\ &\approx 235. \end{aligned}$$

The actual per-device batch size is

$$\begin{aligned} \frac{B}{N} &= \frac{3 \times 10^6}{4096} \\ &\approx 732. \end{aligned}$$

Since $732 > 235$, this configuration is compute bound.

Runtime The ideal mathematical runtime is

$$\begin{aligned} t_{\text{ideal}} &= \frac{6(3 \times 10^6)(13 \times 10^9)}{(4.59 \times 10^{14})(4096)} \\ &\approx 0.12 \text{ s}. \end{aligned}$$

Assuming approximately 40% hardware utilization,

$$\begin{aligned} t_{\text{actual}} &= \frac{0.12}{0.4} \\ &\approx \boxed{0.31 \text{ s}}. \end{aligned}$$

Chapter 6

Training LLaMA

Training LLaMA 3 on TPU: Counting Parameters and FLOPs

i) The number of FLOPs per token is approximately

$$\begin{aligned} F_{\text{token}} &= 6 \times \text{parameters} \\ &= 6(70 \times 10^9) \\ &= 420 \times 10^9 \text{ FLOPs/token.} \end{aligned}$$

ii) Training for 15 trillion tokens requires

$$\begin{aligned} F_{\text{training}} &= (15 \times 10^{12})(420 \times 10^9) \\ &= 6.3 \times 10^{24} \text{ FLOPs.} \end{aligned}$$

iii) At 40% utilization, the mathematical runtime is

$$\begin{aligned} t_{\text{math}} &= \frac{6.3 \times 10^{24}}{(4.59 \times 10^{14})(8960)(0.4)} \\ &= 3.82 \times 10^6 \text{ seconds.} \end{aligned}$$

This is approximately 44 days, or roughly six weeks.

iv) Let the batch contain

$B = 4 \times 10^6$ tokens.

Suppose we store four checkpoints per layer. In particular, we store the activation after attention, the activation after the up projection, and the activation after the down projection: BD, $2BF$, BD. Since the activations use `bfloat16`, the activation memory is

$$M_{\text{activations}} = 2\left(BD + 2BF + BD\right)L$$

$$= 2\,(2B(D+F))\,L.$$

Using $D = 8192, \qquad F = 28672, \qquad L = 80$, we obtain

$$\begin{aligned} M_{\text{activations}} &= 2\left[2(4\times10^6)(8192+28672)\right](80) \\ &\approx 47 \text{ TB}. \end{aligned}$$

The parameters and optimizer state require approximately ten bytes per parameter:

$$\begin{aligned} M_{\text{parameters+optimizer}} &= 10(70\times10^9) \\ &= 700\times10^9 \text{ bytes} \\ &= 700 \text{ GB}. \end{aligned}$$

Therefore, the total memory requirement is approximately $M_{\text{total}} \approx 47.7$ TB. Each TPU contains 96 GB of HBM, so the minimum number of TPUs is approximately

$$\frac{47.7 \text{ TB}}{96 \text{ GB}} \approx 492.$$

In the Chinchilla calculation, all checkpoints are assumed to have shape BD. This gives approximately 225 TPUs, or a little less than half as many.

v) Assuming the total stored memory is 21.6 TB, the memory stored by each of the 8960 chips is

$$\begin{aligned} M_{\text{per chip}} &= \frac{21.6 \text{ TB}}{8960} \\ &\approx \boxed{2.41 \text{ GB}}. \end{aligned}$$

How to Shard LLaMA 3-70B for Training

Assume a batch of

4×10^6 tokens,

consisting of 1024 sequences of length 4096.

No Context Parallelism

If context parallelism is unavailable, the sequence dimension cannot be split.

The computation can therefore only be split 1024 ways using data parallelism. Each chip holds approximately eight times more memory, but this is still less than the available 96 GB of HBM.

Thus, the model fits, but only approximately $\frac{1}{8}$ of the chips are used.

Pure FSDP

From the previous section, pure FSDP is inefficient when the operation is communication bound.

The communication-bound condition is

$$\frac{4 \times 10^6}{8960} < \frac{2550}{3},$$
$$446 < 840.$$

This condition is true. Therefore, the operation is communication bound and pure FSDP is not very efficient.

Mixed FSDP and Tensor Parallelism

The operation remains compute bound provided

$$\frac{4 \times 10^6}{8960} > \frac{2550^2}{2(28672)}.$$

The right-hand side is approximately 113. Since $446 > 113$, mixed FSDP and tensor parallelism can be used.

Let $M_x = 2, \qquad M_y = 1$. Using the previous result,

$$\begin{aligned} X_{\text{opt}} &= \sqrt{\frac{B}{F}\frac{M_x}{M_y}N} \\ &= \sqrt{\frac{(4 \times 10^6)(2)(8960)}{(28672)(1)}} \\ &\approx 1619. \end{aligned}$$

A convenient configuration is 1120-way FSDP and 8-way tensor parallelism, since $1120 \cdot 8 = 8960$.

Q1

To scale training across multiple pods, we must use the data-center network.

The strategy is:

- use pure data parallelism across pods; and

- use FSDP and tensor parallelism within each ICI domain.

Assume approximately

1×10^6 tokens per ICI domain.

The per-chip batch size is approximately 112. This is slightly below the compute-bound threshold of 113. Therefore, the computation is barely communication bound for the local FSDP and tensor-parallel communication, while it is compute bound across pods.

Equivalently, $t_{\text{math,pod}} > t_{\text{comms,DP}}$, for the data-parallel communication between pods, while $t_{\text{comms,local}} \gtrsim t_{\text{math,local}}$ within each pod.

Because the local communication time is very close to the mathematical time, the computation is effectively compute bound and uses the hardware efficiently.

The training time is

$$\begin{aligned} t_{\text{training}} &= \frac{6.3 \times 10^{24}}{(8960)(4)(0.4)(4.59 \times 10^{14})} \\ &\approx 11 \text{ days.} \end{aligned}$$

Thus, training takes approximately $\boxed{\text{11 days at40\% utilization}}$.

Q2

Part (a): Parameter Count

Use

$$\begin{aligned} L &= 126, \\ D &= 16384, \\ F &= 53248, \\ N &= 128, \\ K &= 8, \\ H &= 128, \\ V &= 128256. \end{aligned}$$

The feedforward parameters are approximately

$$\begin{aligned} P_{\text{FFW}} &= 3LDF \\ &\approx 330 \text{ billion.} \end{aligned}$$

The vocabulary parameters are approximately

$$\begin{aligned} P_{\text{vocab}} &= DV \\ &\approx 2 \text{ billion.} \end{aligned}$$

The attention parameters are approximately

$$\begin{aligned} P_{\text{attention}} &= 2LD(N+K)H \\ &\approx 71 \text{ billion.} \end{aligned}$$

Therefore, the total number of parameters is approximately

$$\begin{aligned} P_{\text{total}} &= P_{\text{FFW}} + P_{\text{vocab}} + P_{\text{attention}} \\ &\approx 404 \text{ billion} \\ &\approx \boxed{405 \text{ billion parameters}}. \end{aligned}$$

Part (b): Training Configuration

Assume

$B = 4 \times 10^6$ tokens.

With eight pods, the batch per pod is $B_{\text{pod}} = 500{,}000$ tokens. With context parallelism, the per-chip batch size is approximately

$$\begin{aligned} B_{\text{chip}} &= \frac{468}{8} \\ &\approx 59. \end{aligned}$$

This is well below the pure-FSDP threshold of 840. Therefore, pure FSDP is not appropriate.

For mixed FSDP and tensor parallelism, we require approximately

$$\begin{aligned} 59 &> \frac{2550^2}{2F} \\ &\approx 61. \end{aligned}$$

This condition is not strictly satisfied, but it is close enough that we can expect good utilization.

We also have $500{,}000 > 80{,}000$, so the configuration is acceptable with respect to the DCN communication threshold.

Putting everything together, the expected configuration is:

- data parallelism across pods; and
- mixed FSDP and tensor parallelism within each pod.

The training time is

$$\begin{aligned} t_{\text{training}} &= \frac{6(15 \times 10^{12})(405 \times 10^{9})}{(4.59 \times 10^{14})(8960)(8)(0.4)} \\ &= 2.8 \times 10^{6} \text{ seconds.} \end{aligned}$$

Therefore, training takes approximately $\boxed{\text{32 days at40\% utilization}}$.

Chapter 7

Inference

Q1

Assume two weight matrices for attention and one output weight matrix per layer.

The MLP contains

$$P_{\text{MLP}} = 64(4096)(16384)(3).$$

The attention layers contain

$$P_{\text{attention}} = 64\left[2(8)(256)(4096) + 2(32)(256)(4096)\right].$$

The embedding and shared projection parameters contain

$$P_{\text{embedding}} = (32128)(4096).$$

The resulting model size is approximately $\boxed{18.4\text{ GB}}$. For each token, the KV cache contains keys and values for every layer, KV head, and head dimension:

$$\begin{aligned} M_{\text{KV/token}} &= 2(64)(8)(256)(2) \\ &= 262144 \text{ bytes} \\ &\approx \boxed{262\text{ KB}}. \end{aligned}$$

Q2

For a sequence length of 128K, the KV cache requires

$$\begin{aligned} M_{\text{KV}} &= 128\text{K} \cdot 262 \text{ KB} \\ &\approx 33.5 \text{ GB}. \end{aligned}$$

The model itself requires 18.4 GB. Assume the model is fully sharded across the 16 chips. The memory per chip is

$$\begin{aligned} M_{\text{per chip}} &= \frac{33.5 \text{ GB} + 18.4 \text{ GB}}{16} \\ &\approx 3.24 \text{ GB}. \end{aligned}$$

We also need enough space for the attention scores.

At a maximum sequence length of 128K, the attention-score tensor contains approximately $128\text{K} \cdot 256 \cdot 32$ values, requiring approximately 16.3 GB. With 256 GB of total memory, the maximum batch size is

$$\begin{aligned} B_{\text{max}} &= \frac{256 \text{ GB} - 18.4 \text{ GB}}{33.5 \text{ GB} + 16.3 \text{ GB}} \\ &\approx 4.7. \end{aligned}$$

The calculation must also reserve space for buffering activations. Therefore, the runtime values near $B = 7$ are borderline.

Q3

The TPUv5e HBM bandwidth is $W_{\text{HBM}} = 8.2 \times 10^{11}$ bytes/s. To load an 18.4 GB parameter model distributed across 16 chips, the required time is

$$\begin{aligned} t_{\text{load}} &= \frac{18.4 \times 10^9}{(8.2 \times 10^{11})(16)} \\ &\approx \boxed{1.4 \text{ ms}}. \end{aligned}$$

Q4

We have a 4×4 chip array, so there are no wrap-around links. This limits the communication speed.

An AllGather would normally use the full bidirectional bandwidth, but without wrap-around links, data may require six hops to travel from one corner to the opposite corner.

The communication time is approximated by

$$\begin{aligned} t_{\text{comms}} &= 2\left(\frac{V}{16}\right)(6)\left(\frac{1}{4.5 \times 10^{10}}\right) \\ &= \frac{12BD}{16(4.5 \times 10^{10})}. \end{aligned}$$

The mathematical time is

$$t_{\text{math}} = \frac{4BDF}{16(3.94 \times 10^{14})}.$$

For the operation to be compute bound, we require

$$\begin{aligned} t_{\text{math}} &> t_{\text{comms}}, \\ \frac{4BDF}{16(3.94 \times 10^{14})} &> \frac{12BD}{16(4.5 \times 10^{10})}. \end{aligned}$$

Cancelling the common factors gives approximately

$$\begin{aligned} \frac{4F}{12} &> 8500, \\ F &> 25500. \end{aligned}$$

Since $F = 16384$, the operation is never compute bound with 16-way model parallelism.

If we instead use 4-way sharding, there are only three hops. The corresponding condition becomes

$$\begin{aligned} \frac{4}{6}F &> 8500, \\ F &> 12750. \end{aligned}$$

Since $16384 > 12750$, the operation is compute bound.

Therefore, the takeaway is Use 4-way tensor parallelism during prefill.

KV-Cache Sharding

For the KV cache, pure Megatron-style sharding can be used up to 8-way, corresponding to the number of KV heads.

During prefill, use

- 4-way model parallelism across Y; and
- 4-way sequence parallelism across X.

During generation, use 16-way model parallelism.

The HBM and ICI times are compared using

$$\frac{12BD}{16(4.5 \times 10^{10})} > \frac{DF}{(8.2 \times 10^{11})(16)}.$$

Equivalently,

$$\frac{F}{12B} < \frac{8.2 \times 10^{11}}{4.5 \times 10^{10}}.$$

For $F = 16384$, the corresponding batch-size transition is approximately $B \approx 75$. The same KV-cache sharding is used during prefill.

The ICI communication consists of an AllReduce over X and an AllToAll that changes the sharding axis:

$$\begin{aligned} t_{\text{KV,ICI}} &= \frac{2BD}{W_{\text{ICI}}} + \frac{2\left(\frac{BD}{2}\right)\left(\frac{1}{4}\right)}{W_{\text{ICI}}} \\ &= \frac{2BD + \frac{BD}{4}}{W_{\text{ICI}}} \\ &= \frac{2.25BD}{4(4.5 \times 10^{10})} \\ &\approx 1.0 \times 10^{-6} \text{ s}. \end{aligned}$$

The HBM cost of reading the KV cache is

$$t_{\text{KV,HBM}} = \frac{B(33.5 \text{ GB})}{(8.2 \times 10^{11})(16)}.$$

For $B = 7$, this is approximately $t_{\text{KV,HBM}} \approx 17$ ms. The HBM cost therefore lower-bounds the generation time.

The minimum theoretical step time is

$$\begin{aligned} t_{\text{step}} &\approx 17 \text{ ms} + \frac{18.4 \text{ GB}}{(8.2 \times 10^{11})(16)} \\ &\approx 17 \text{ ms} + 1.4 \text{ ms} \\ &= \boxed{18.4 \text{ ms}}. \end{aligned}$$

Q5

Part 1: Parameter Counts

The attention parameter count is

$$\begin{aligned} P_{\text{attention}} &= L\left[2(DHN) + 2(DHK)\right] \\ &= 64\left[2(4096)(256)(32) + 2(4096)(256)(8)\right] \\ &\approx 5.37 \times 10^9. \end{aligned}$$

The embedding and unembedding contain

$$\begin{aligned} P_{\text{embedding}} &= DV \\ &= (4096)(32128) \\ &\approx 131 \times 10^6. \end{aligned}$$

The MLP parameters are

$$\begin{aligned} P_{\text{MLP}} &= LE(3DF) \\ &= (64)(8)(3)(4096)(16384) \\ &\approx 206 \times 10^9. \end{aligned}$$

Thus, the total parameter count is approximately $P_{\text{total}} \approx 211$ billion. Only one of the eight experts is activated for each token. Therefore, the activated parameter count is

$$\begin{aligned} P_{\text{activated}} &= \frac{206}{8} + 5.37 + 0.131 \\ &\approx \boxed{30 \text{ billion}}. \end{aligned}$$

Part 2: Critical Batch Size

Assume that, for a particular batch, all experts are loaded into memory.

This requires loading approximately 16 times more parameters than before, while the amount of compute increases by approximately a factor of 2, assuming the MLP dominates.

The original critical batch size was $B_{\text{crit}} = 240$. The new memory-to-compute ratio is worse by a factor of $\frac{16}{2} = 8$. Therefore,

$$\begin{aligned} B_{\text{crit,MoE}} &= 8(240) \\ &\approx \boxed{1900}. \end{aligned}$$

Part 3: KV Cache

The KV caches are identical to those of the dense model because the MoE architecture does not change the attention portion.

Part 4: FLOPs per Token

A full training pass requires approximately $6PT$ FLOPs, while a forward pass requires approximately $2PT$. Since only the activated experts contribute to the forward computation, $P_{\text{activated}} \approx 30$ billion. Therefore, the forward-pass FLOP count is

$$\begin{aligned} F_{\text{forward}} &= 2P_{\text{activated}}T \\ &= 2(30 \times 10^9)T \\ &= \boxed{60 \times 10^9 T}. \end{aligned}$$

Q6

Part 1

Shard the model as $[F_Z, D, F_Y]$. The total parameter count is approximately 210 billion, and is dominated by the MLP weights.

The time required to load the model is

$$\begin{aligned} t_{\text{load}} &= \frac{(210 \times 10^9)(2)}{(8.2 \times 10^{11})(16)(8)} \\ &\approx \boxed{2 \text{ ms}}. \end{aligned}$$

Each chip stores approximately 1.64 GB of weights.

With 16 GB of HBM per chip, this leaves approximately

$$16 \text{ GB} - 1.64 \text{ GB} = 14.36 \text{ GB}$$

available per chip.

Part 2

The model occupies approximately 210 GB. A TPUv5e contains 16 GB per chip.

The closest clean power-of-two mesh that fits the model is 16×16. A slightly smaller arrangement such as 16×14 could also fit, using approximately $Z = 16, \quad Y = 14$.

Q7

Assume bidirectional communication.

Consider two fully sharded matrix multiplications. Since all matrix multiplications are fully sharded, there is no replicated compute.

The mathematical runtime is

$$\begin{aligned} t_{\text{math}} &= t(\text{In} \cdot W_{\text{in}}) + t(\text{Temp} \cdot W_{\text{out}}) \\ &= \frac{2BDF}{P_{\text{chip}}} + \frac{2BDF}{P_{\text{chip}}} \\ &= \frac{4BDF}{P_{\text{chip}}}. \end{aligned}$$

Collective Communication

The communication consists of

$$\begin{aligned} t_{\text{comms}} = &\text{AllGather}_{YZ}\left(\text{In}[B, D_{XYZ}]\right) \\ &+ \text{AllReduce}_{X}\left(\text{Temp}[B, F_{YZ}]\{0_X\}\right) \\ &+ \text{ReduceScatter}_{YZ}\left(\text{Out}[B, D_X]\{0_{YZ}\}\right). \end{aligned}$$

The corresponding communication time is

$$\begin{aligned} t_{\text{comms}} &= \frac{2BD}{2XW_{\text{ICI}}} + \frac{4BF}{YZW_{\text{ICI}}} + \frac{2BD}{2XW_{\text{ICI}}} \\ &= \frac{4BD}{2XW_{\text{ICI}}} + \frac{4BF}{YZW_{\text{ICI}}}. \end{aligned}$$

Sharding over D_X and F_{YZ} attempts to make the local chunks approximately square. It is therefore reasonable to compare X with YZ.

First assume $F = 4D$. Then

$$t_{\text{comms}} = \frac{4BD}{2XW_{\text{ICI}}} + \frac{16BD}{YZW_{\text{ICI}}}.$$

Let $N = XYZ$. Since $YZ = \frac{N}{X}$, we obtain

$$t_{\text{comms}} = \frac{4BD}{2XW_{\text{ICI}}} + \frac{16BD}{\left(\frac{N}{X}\right)W_{\text{ICI}}}$$

$$= \frac{4BD}{2XW_{\text{ICI}}} + \frac{16BDX}{NW_{\text{ICI}}}.$$

Comparing this with communication proportional to $\frac{4BD}{W_{\text{ICI}}}$, the comparison reduces to

$$\frac{4BD}{2X} + \frac{16BDX}{N} < 4BD.$$

Using a common denominator,

$$\frac{4BDN + 32BDX^2}{2XN} < 4BD.$$

Cancelling $4BD$,

$$\frac{N + 8X^2}{2XN} < 1.$$

Generalization to Arbitrary F

For arbitrary F,

$$t_{\text{comms}} = \frac{4BD}{2XW_{\text{ICI}}} + \frac{4BFX}{NW_{\text{ICI}}}.$$

Comparing this with the unsharded expression gives

$$\frac{4BD}{2X} + \frac{4BFX}{N} < 4BD.$$

Using a common denominator,

$$\frac{4BDN + 8BFX^2}{2XN} < 4BD.$$

Equivalently,

$$\frac{4B(DN + 2FX^2)}{2XN} < 4BD,$$

so

$$\frac{DN + 2FX^2}{2XN} < D.$$

Optimal Mesh Shape

For fixed N, there is a value of X and YZ that minimizes the communication time.

Ignoring constants and W_{ICI},

$$t_{\text{comms}}(X) = \frac{4BD}{2X} + \frac{4BFX}{N}.$$

The first term decreases monotonically with X, while the second term increases monotonically with X.

The derivative is

$$\frac{d}{dX} t_{\text{comms}} = -\frac{2BD}{X^2} + \frac{4BF}{N}.$$

The optimum occurs when

$$\frac{2BD}{X^2} = \frac{4BF}{N}.$$

Cancelling $2B$,

$$\frac{D}{X^2} = \frac{2F}{N}.$$

Therefore,

$$X^2 = \frac{DN}{2F},$$

$$\boxed{X_{\text{opt}} = \sqrt{\frac{DN}{2F}}}.$$

Since $YZ = \frac{N}{X}$, the corresponding product of the other two mesh dimensions is

$$YZ_{\text{opt}} = \frac{N}{\sqrt{\frac{DN}{2F}}}.$$

Substitution into the Communication Criterion

The criterion for the three-dimensional sharding to improve communication is

$$\frac{DN + 2FX^2}{2XN} < D.$$

Substituting $X = \sqrt{\frac{DN}{2F}}$, gives

$$\frac{DN + 2F\left(\sqrt{\frac{DN}{2F}}\right)^2}{2N\sqrt{\frac{DN}{2F}}} < D.$$

Squaring the expression gives the intermediate condition

$$\frac{\left[DN + 2F\left(\sqrt{\frac{DN}{2F}}\right)^2\right]^2}{4N^2\left(\frac{DN}{2F}\right)} < D^2.$$

The numerator expands into terms of the form

$$D^2N^2 + 4DFN\sqrt{\frac{DN}{2F}} + \frac{DN}{2F}.$$

The resulting inequality can also be written as

$$\frac{DFN}{2} + 2F^2\sqrt{\frac{DN}{2F}} + \frac{1}{4} < D^2N^2.$$

Simplified Threshold

Let $F = \alpha D$. The communication time becomes

$$t_{\text{comms}} = \frac{4BD}{2X} + \frac{4B\alpha DX}{N}.$$

We want this to be less than $\frac{4BD}{3}$. Thus,

$$\frac{4BD}{2X} + \frac{4B\alpha DX}{N} < \frac{4BD}{3}.$$

Cancelling $4BD$,

$$\frac{1}{2X} + \frac{\alpha X}{N} < \frac{1}{3}.$$

For $F = \alpha D$, the optimal mesh dimension is

$$\begin{aligned} X_{\text{opt}} &= \sqrt{\frac{DN}{2F}} \\ &= \sqrt{\frac{N}{2\alpha}}. \end{aligned}$$

The inequality can be written as

$$\frac{N + 2\alpha X^2}{2XN} < \frac{1}{3}.$$

Substituting $X^2 = \frac{N}{2\alpha}$, gives

$$\frac{N + 2\alpha\left(\frac{N}{2\alpha}\right)}{2XN} < \frac{1}{3}.$$

Therefore,

$$\begin{aligned} \frac{2N}{2XN} &< \frac{1}{3}, \\ \frac{1}{X} &< \frac{1}{3}, \\ X &> 3. \end{aligned}$$

Using $X = \sqrt{\frac{N}{2\alpha}}$, we require

$$\begin{aligned} \sqrt{\frac{N}{2\alpha}} &> 3, \\ \frac{N}{2\alpha} &> 9, \\ N &> 18\alpha. \end{aligned}$$

Since $\alpha = \frac{F}{D}$, the final condition is $\boxed{N > 18\frac{F}{D}}$. For $F = 4D$, this becomes

$$\begin{aligned} N &> 18(4) \\ &= \boxed{72 \text{ chips}}. \end{aligned}$$

Chapter 8

Serving LLaMA

Basic Quantities

Q1

Let $K = 8$, $H = 128$, $L = 80$. Using `int8` for the KV cache, each value occupies one byte. The KV-cache size per token is therefore

$$\begin{aligned} M_{\text{KV/token}} &= 2KHL \cdot 1 \text{ byte} \\ &= 2(8)(128)(80) \text{ bytes} \\ &= 163{,}840 \text{ bytes} \\ &\approx \boxed{164 \text{ KB}}. \end{aligned}$$

Q2

Let $B = 32$, $T = 8192$, and use `int8` quantization.

The total KV-cache memory is

$$\begin{aligned} M_{\text{KV}} &= BT\,(164 \text{ KB}) \\ &= (32)(8192)\,(164 \text{ KB}) \\ &\approx 43 \text{ GB}. \end{aligned}$$

The parameter memory is $M_{\text{parameters}} = 70$ GB. Thus, the total memory requirement is

$$\begin{aligned} M_{\text{total}} &= 43 \text{ GB} + 70 \text{ GB} \\ &= \boxed{113 \text{ GB}}. \end{aligned}$$

A TPUv5e chip contains 16 GB of HBM, so the minimum number of chips is

$$\frac{113}{16} = 7.06.$$

Therefore, we need approximately eight chips, which can be arranged as $\boxed{4 \times 2}$.

Q3

Assuming that the operation is memory bound, the latency is

$$\begin{aligned} t_{\text{latency}} &= \frac{M_{\text{KV}} + M_{\text{parameters}}}{NW_{\text{HBM}}} \\ &= \frac{43 \text{ GB} + 70 \text{ GB}}{(8)(8.2 \times 10^{11})} \\ &\approx \boxed{17.2 \text{ ms}}. \end{aligned}$$

The total token throughput is

$$\begin{aligned} \text{Throughput} &= \frac{B}{t_{\text{latency}}} \\ &= \frac{32}{17.2 \text{ ms}} \\ &\approx 1.86 \text{ tokens/ms} \\ &= 1860 \text{ tokens/s}. \end{aligned}$$

The throughput per chip is

$$\begin{aligned} \text{Throughput}_{\text{chip}} &= \frac{1.86}{8} \text{ tokens/ms} \\ &= 0.231 \text{ tokens/ms/chip} \\ &= \boxed{231 \text{ tokens/s/chip}}. \end{aligned}$$

If we instead use a 4×4 slice, the latency is approximately

$$\begin{aligned} t_{\text{latency}} &= \frac{113 \text{ GB}}{(16)(8.2 \times 10^{11})} \\ &= \frac{17.2 \text{ ms}}{2} \\ &\approx \boxed{8.6 \text{ ms}}. \end{aligned}$$

The total throughput therefore increases from 1860 to $\boxed{3720 \text{ tokens/s}}$.

Thinking About Throughput

Q1

For TPUv5e with `bfloat16`, the critical arithmetic intensity is approximately 240. If the parameters are stored in `int8`, the required HBM traffic is divided by two. The critical value therefore becomes

$$240 \longrightarrow 120.$$

If the parameters are both stored and computed in `int8`, then the HBM traffic and the compute cost are both reduced by a factor of two. Therefore, the ratio does not change:

$$\boxed{240}.$$

Q2

Consider a 70-billion-parameter model.

With `bfloat16` parameters, the model occupies 140 GB. The required number of TPUv5e chips is

$$\frac{140}{16} = 8.75.$$

Therefore, we need a 4×4 slice.

With `int8` parameters, the model occupies 70 GB, so

$$\frac{70}{16} = 4.4.$$

Therefore, we need a 4×2 slice.

With `int4` parameters, the model occupies 35 GB, so

$$\frac{35}{16} = 2.18.$$

A 2×2 slice is sufficient.

KV-Cache Memory per Sequence

For a sequence length of $T = 8192$, the `bfloat16` KV-cache memory per sequence is

$$\begin{aligned} M_{\text{KV,bf16}} &= 164 \text{ KB} \cdot 2 \cdot 8192 \\ &\approx \boxed{2.7 \text{ GB}}. \end{aligned}$$

For `int8`,

$$\begin{aligned} M_{\text{KV,int8}} &= \frac{2.7 \text{ GB}}{2} \\ &= \boxed{1.35 \text{ GB}}. \end{aligned}$$

For `int4`,

$$\begin{aligned} M_{\text{KV,int4}} &= \frac{1.35 \text{ GB}}{2} \\ &\approx \boxed{0.672 \text{ GB}}. \end{aligned}$$

Q3

We first calculate the largest batch that fits during prefill.

`bfloat16`

A 4×4 slice contains $16(16 \text{ GB}) = 256$ GB of HBM.

After storing the 140 GB model, the remaining KV-cache memory is $256 - 140 = 116$ GB. Since each sequence uses 2.7 GB,

$$\begin{aligned} B_{\max} &= \frac{116}{2.7} \\ &\approx 42.9. \end{aligned}$$

`int8`

A 4×2 slice contains $8(16 \text{ GB}) = 128$ GB. The remaining memory is $128 - 70 = 58$ GB. Thus,

$$\begin{aligned} B_{\max} &= \frac{58}{1.35} \\ &\approx 42.9. \end{aligned}$$

`int4`

A 2×2 slice contains $4(16\text{ GB}) = 64\text{ GB}$. The remaining memory is $64 - 35 = 29\text{ GB}$. Thus,

$$\begin{aligned} B_{\max} &= \frac{29}{0.672} \\ &\approx 43. \end{aligned}$$

Therefore, for every quantization level, the maximum practical batch size is approximately $\boxed{B = 42}$.

Latency

For `bfloat16`,

$$\begin{aligned} t_{\text{bf16}} &= \frac{140\text{ GB} + 42(2.7\text{ GB})}{(16)(8.2 \times 10^{11})} \\ &\approx 19\text{ ms}. \end{aligned}$$

For `int8`,

$$\begin{aligned} t_{\text{int8}} &= \frac{70\text{ GB} + 42(1.35\text{ GB})}{(8)(8.2 \times 10^{11})} \\ &\approx 19\text{ ms}. \end{aligned}$$

For `int4`,

$$\begin{aligned} t_{\text{int4}} &= \frac{35\text{ GB} + 42(0.672\text{ GB})}{(4)(8.2 \times 10^{11})} \\ &\approx 19\text{ ms}. \end{aligned}$$

The KV cache, parameter memory, and number of chips are all divided by two at each quantization level, so the latency remains approximately the same.

Q4

The total token throughput is

$$\text{Throughput}_{\text{tokens}} = \frac{B}{t_{\text{latency}}}$$

$$
\begin{aligned}
&= \frac{42}{0.019} \\
&\approx \boxed{2210 \text{ tokens/s}}.
\end{aligned}
$$

`bfloat16`

With 16 chips, the token throughput per chip is

$$\frac{2210}{16} \approx 138 \text{ tokens/s/chip}.$$

Assuming a median decode length of 512,

$$
\begin{aligned}
\text{Throughput}_{\text{queries/chip}} &= \frac{138}{512} \\
&\approx \boxed{0.27 \text{ queries/s/chip}}.
\end{aligned}
$$

`int8`

With 8 chips,

$$
\begin{aligned}
\text{Throughput}_{\text{queries/chip}} &= \frac{2210}{(8)(512)} \\
&\approx \boxed{0.54 \text{ queries/s/chip}}.
\end{aligned}
$$

`int4`

With 4 chips,

$$
\begin{aligned}
\text{Throughput}_{\text{queries/chip}} &= \frac{2210}{(4)(512)} \\
&\approx \boxed{1.09 \text{ queries/s/chip}}.
\end{aligned}
$$

Q5

The memory available for the KV cache is $M_{\text{KV,available}} = M_{\text{HBM}} - M_{\text{parameters}}$. Therefore,

$$B = \frac{M_{\text{HBM}} - M_{\text{parameters}}}{M_{\text{KV/sequence}}}.$$

Moving from `bfloat16` to `int8`, and then from `int8` to `int4`, divides both the model memory and the KV-cache memory by two. Consequently, the maximum batch size remains approximately the same for each quantization level.

Q6

Suppose the available HBM is doubled.

For `bfloat16`, the maximum batch becomes

$$\begin{aligned} B_{\max} &= \frac{512 \text{ GB} - 140 \text{ GB}}{2.7 \text{ GB}} \\ &\approx 137.8. \end{aligned}$$

Thus, use approximately $B = 137$. The query throughput per chip is

$$\begin{aligned} \text{Throughput}_{\text{bf16}} &= \frac{137}{(0.019)(32)(512)} \\ &\approx \boxed{0.44 \text{ queries/s/chip}}. \end{aligned}$$

The corresponding throughputs are approximately $\boxed{0.88 \text{ queries/s/chip}}$ for `int8`, and $\boxed{1.76 \text{ queries/s/chip}}$ for `int4`.

Model-Parallelism Constraint

During generation, we can only use model parallelism. On a 4×8 slice, this gives $4 \cdot 8 = 32$ ways of model parallelism.

Using $F = 28672$, we have

$$\begin{aligned} 32 &> \frac{2F}{2500} \\ &= \frac{2(28672)}{2500} \\ &\approx 23. \end{aligned}$$

Therefore, this configuration is not compute bound.

The HBM-to-ICI threshold is

$$\frac{F}{B\,(W_{\text{HBM}}/W_{\text{ICI}})} = \frac{28672}{137\,(8.2 \times 10^{11}/9.0 \times 10^{10})}$$

$$\approx 23.$$

Since the model-parallel degree is 32, we still incur some inefficiency.

If we reduce the batch to $B = 24$, the corresponding threshold is approximately 131. We can instead use $B = 48$, which gives a threshold of approximately 65. Since $32 < 65$, this configuration can remain HBM bound and avoid being ICI bound.

If we use an even larger slice, such as $4 \times 4 \times 8$, we can use two-dimensional weight-stationary sharding and a hypercube topology to obtain better hardware utilization.

Prefill

Q1

Let the prefill length be

$T = 8192$ tokens.

Using 16 TPUv5e chips, the per-chip batch is $B = 512$. Since $512 > 240$, the prefill operation is compute bound.

A forward pass requires approximately

$$\begin{aligned} F_{\text{prefill}} &= 2TP \\ &= 2(8192)(70 \times 10^9) \\ &\approx 1.146 \times 10^{15} \text{ FLOPs.} \end{aligned}$$

At 100% utilization,

$$\begin{aligned} t_{\text{prefill}} &= \frac{1.146 \times 10^{15}}{(1.97 \times 10^{14})(16)} \\ &\approx 0.36 \text{ s.} \end{aligned}$$

At 40% utilization,

$$\begin{aligned} t_{\text{prefill}} &= 0.36 \left(\frac{10}{4} \right) \\ &\approx \boxed{0.904 \text{ s}}. \end{aligned}$$

Q2

Assume median prefill length = 8192, median decode length = 4096,

and $B = 32$. One decode step generates 32 tokens. Therefore, it completes

$$\frac{32}{4096} = \frac{1}{128}$$

of a sequence per step.

Each completed sequence occupies

$$8192 + 4096 = 12288$$

tokens in the KV cache.

Thus, the number of KV-cache tokens evicted per decode step is

$$\frac{12288}{128} = \boxed{96 \text{ tokens}}.$$

Q3

Consider disaggregated serving with separate prefill and generation servers: $\boxed{\text{Prefill}} \longrightarrow \boxed{\text{Generation}}$. The throughput is $\text{Throughput} = B \cdot t_{\text{latency}}^{-1}$. To utilize both systems equally, the prefill servers must produce sequences at the same rate at which the generation servers consume them.

Assume generation uses $B = 32$ and the median decode length is 512. Each decode step generates 32 tokens, so a sequence requires

$$\frac{512}{32} = 16$$

decode steps.

If each generation step takes 19 ms, then the generation time per sequence is

$$\begin{aligned} t_{\text{generation/sequence}} &= 16(19 \text{ ms}) \\ &= \boxed{304 \text{ ms}}. \end{aligned}$$

The prefill server processes one sequence in approximately 904 ms. Therefore,

$$\frac{t_{\text{prefill}}}{t_{\text{generation}}} = \frac{904}{304}$$

$$\approx 3.$$

The prefill server takes approximately three times as long to produce a sequence as the generation server takes to consume one. Therefore, we need roughly 3 prefill servers per generation server.

Worked Problems

Q1

A forward pass requires approximately $2 \times \text{parameters} \times \text{tokens}$. For a 405-billion-parameter model, this is

$$\begin{aligned} F_{\text{token}} &= 2(405 \times 10^9) \\ &= 810 \times 10^9 \text{ FLOPs/token.} \end{aligned}$$

With N TPUv5e chips, the compute-bound latency is

$$\begin{aligned} t_{\text{compute}} &= \frac{810 \times 10^9}{N(1.97 \times 10^{14})} \\ &\approx \boxed{\frac{4}{N} \text{ ms}}. \end{aligned}$$

Assuming `int8` parameter storage, the communication-bound latency is

$$\begin{aligned} t_{\text{comms}} &= \frac{405 \times 10^9}{N(8.2 \times 10^{11})} \\ &\approx \boxed{\frac{494}{N} \text{ ms}}. \end{aligned}$$

Q2

Parameter Memory

With `int8` storage, the model parameters require $\boxed{405 \text{ GB}}$.

KV-Cache Memory

For a 32K-token context, the KV cache requires approximately

$$\begin{aligned} M_{\text{KV}} &= 240\,(8 \cdot 128 \cdot 32\text{K}) \\ &\approx \boxed{7.86 \text{ GB}}. \end{aligned}$$

Maximum Intermediate Activation

The maximum intermediate activation size is approximately

$$\begin{aligned} M_{\text{intermediate}} &= 240\text{K} \cdot F \\ &= 240\text{K} \cdot 14336 \\ &\approx \boxed{3.44 \text{ GB}}. \end{aligned}$$

For serving, assume an 8192-token median prefill length.

Q3

The model occupies 405 GB with `int8` storage.

The minimum chip count is approximately

$$\frac{405}{16} \approx 25.$$

Therefore, we need at least a 4×8 slice.

The total HBM on 32 chips is $32(16 \text{ GB}) = 512$ GB. After storing the parameters, the remaining KV-cache memory is

$$512 - 405 = 107 \text{ GB}.$$

The KV-cache memory per sequence is

$$\begin{aligned} M_{\text{KV/sequence}} &= 2(126)(128)(8)(8192) \\ &\approx \boxed{2.11 \text{ GB}}. \end{aligned}$$

Therefore, the batch size is approximately

$$B = \frac{107}{2.11}$$

$$\approx 48.$$

The resulting latency is

$$\begin{aligned} t_{\text{latency}} &= \frac{405 \text{ GB} + 48(2.11 \text{ GB})}{(32)(8.2 \times 10^{11})} \\ &\approx 19 \text{ ms}. \end{aligned}$$

This is too high.

Increasing the Slice to $4 \times 4 \times 4$

A $4 \times 4 \times 4$ slice contains 64 chips. Unfortunately, this is ICI-bound since we are doing 64-way TP. If we imagined we were not, then the math would look as follows. A clever solution could be to 2-D model shard, which we explore after.

The maximum batch is approximately $B_{\text{max}} \approx 290$. At $B = 160$, the lower bound on the linear-layer latency is approximately 14 ms. The mathematical runtime is

$$\begin{aligned} t_{\text{math}} &= \frac{2(405 \times 10^9)(160)}{(1.97 \times 10^{14})(64)} \\ &\approx 10.2 \text{ ms}. \end{aligned}$$

The KV-cache loading time is

$$\begin{aligned} t_{\text{KV}} &= \frac{160(2.11 \text{ GB})}{(8.2 \times 10^{11})(64)} \\ &\approx 6 \text{ ms}. \end{aligned}$$

Thus, the total latency is approximately $10.2 \text{ ms} + 6 \text{ ms} = 16.2 \text{ ms}$, which is still too high.

If we reduce the batch to $B = 140$, the linear-layer latency is approximately 9 ms, and the KV-cache loading time is approximately 5.6 ms. Therefore, $9 \text{ ms} + 5.6 \text{ ms} = 14.6 \text{ ms}$, which is still greater than the target latency.

Two-Dimensional Model Parallelism

We can instead use a $4 \times 8 \times 4$ slice and apply two-dimensional model parallelism.

The ICI communication has the form

$$t_{\text{ICI}} \propto \frac{4BD}{2X} + \frac{4B\alpha DX}{N},$$

where $\alpha = \frac{F}{D}$. Using the coefficient from the calculation,

$$t_{\text{ICI}} \propto \frac{4BD}{2X} + \frac{12.75BDX}{N}.$$

The optimal value of X is

$$\begin{aligned} X_{\text{opt}} &= \sqrt{\frac{N}{2\alpha}} \\ &\approx 4. \end{aligned}$$

For $N = 128$ and $X = 4$, the communication expression becomes approximately

$$\frac{BD}{2} + \frac{50BD}{128} \lesssim BD.$$

Avoiding ICI-Boundedness

The ICI time is bounded by $t_{\text{ICI}} \lesssim \frac{BD}{W_{\text{ICI}}}$. To remain HBM bound, require

$$\frac{W_{\text{HBM}}}{W_{\text{ICI}}} > \frac{2F}{NB}.$$

Let

$\beta = \frac{W_{\text{HBM}}}{W_{\text{ICI}}} \approx 8$. Then

$$N > \frac{2F}{\beta B}.$$

For $B = 120$, this requires approximately $N > 128$. Thus, with approximately 128 chips, two-dimensional model parallelism can avoid being ICI bound.

Final Generation Latency

Using $B = 120$ and $N = 128$, the generation latency is

$$\begin{aligned} t_{\text{generation}} &= \frac{120(2.11 \text{ GB}) + 405 \text{ GB}}{(128)(8.2 \times 10^{11})} \\ &\approx \boxed{6 \text{ ms}}. \end{aligned}$$

The token throughput is

$$\begin{aligned}\text{Throughput} &= \frac{120}{6 \text{ ms}} \\ &\approx \boxed{19{,}725 \text{ tokens/s}}.\end{aligned}$$

Prefill Latency

For an 8192-token prefill, the latency is

$$\begin{aligned}t_{\text{prefill}} &= \frac{8192 \cdot 2 \cdot 405 \times 10^9}{(1.97 \times 10^{14})(128)} \\ &\approx \boxed{263 \text{ ms}}.\end{aligned}$$

Balancing Prefill and Generation Servers

One prefill server produces approximately $\frac{1}{263 \text{ ms}}$ sequences per second. The generation server consumes sequences at a rate determined by $\frac{120}{(6 \text{ ms})(8192)}$. The ratio of required prefill servers to generation servers is approximately $\frac{P}{G} \approx 0.64$, or equivalently $\frac{G}{P} \approx 1.5$. A convenient integer configuration is therefore 3 generation servers and 2 prefill servers.

Conclusion

Assuming an 8K median prefill and generation length, use a $4 \times 8 \times 4$ TPUv5e slice with two-dimensional model sharding, 3 generation servers, and 2 prefill servers. This gives a decode per-token latency of approximately $\boxed{6 \text{ ms}}$, a prefill sequence latency of approximately $\boxed{263 \text{ ms}}$, and a decode throughput of approximately $\boxed{19{,}725 \text{ tokens/s}}$.

Chapter 9

Profiling

Q1

The framework NameScope shows that we have the two einsums `fw,bw -> bf` and `bw,bf -> bw`. We can also see a reduction and an AllReduce.

The first einsum lowers locally to the matrix multiplication $[4096] = [4096, 8192] \,@\, [8192]$. Thus, locally, $B = 1, \quad F = 4096, \quad D = 8192$. The copy operation then loads the output. We also see that eight devices participate in the AllReduce.

Putting this together, the first matrix multiplication has the form $[B, D] \,@\, [D, F_X] \longrightarrow [B, F_X]$. The second matrix multiplication contracts over the sharded F dimension: $[B, F_X] \,@\, [F_X, D] \longrightarrow [B, D]\{0_X\}$. This is followed by $\text{AllReduce}_X\left([B, D]\{0_X\}\right) \longrightarrow [B, D]$. This is tensor parallelism.

Since we have eight-way tensor parallelism, $B = 1, \quad D = 8192, \quad F = 32768$. The local F dimension is therefore $\frac{32768}{8} = 4096$, which agrees with the matrix multiplication shown in the profile.

The initial reduction appears to be squeezing the batch dimension.

Q2

Profile Timing

The profile gives the following approximate times:

- The initial embedding takes approximately 11 ms.
- Each transformer layer takes approximately 36 ms.

  Across eight layers, this gives approximately 8(36 ms) = 288 ms.
- The unembedding takes approximately 41 ms.

Adding these together gives a total runtime of approximately 330 ms.

AllGather Before the Unembedding

Zooming in, we observe an AllGather prior to the embedding or unembedding operation. This appears highly wasteful.

From the HLO operation, the relevant dimensions are $B = 8, \quad T = 1024, \quad V = 32128, \quad D = 2048$. The AllGather operation shows that the collective is performed over the B dimension. Consequently, XLA gathers the large unembedded array $[B, 1024, 32128]$. This means that the large vocabulary-sized output is gathered across the batch dimension.

Feedforward Sharding

In the feedforward network, we see an AllReduce following the up projection.

This implies that the contracting dimension D is sharded over X. The profile shows two replica groups.

The local up projection has the form $[B, D_X] \,@\, [D_X, F] \longrightarrow [B, F]\{0_X\}$. It is followed by $\text{AllReduce}_X\,([B, F]\{0_X\}) \longrightarrow [B, F]$. The down projection then has the form $[B, F] \,@\, [F, D_X] \longrightarrow [B, D_X]$.

Attention Sharding

During attention, we also see an AllReduce after the Q, K, and V matrix multiplications.

This implies that D is again sharded over X. Schematically, we have an activation $[B, D_X]$ and projection weights of the forms $[D_X, H, V], \quad [D_X, H, Q], \quad [D_X, H, K]$. Each projection initially produces a partial result over X, which is then combined using an AllReduce.

We also see an AllReduce after W_O. The replica groups have size two here, indicating that the head or value dimensions are sharded over the second mesh axis.

From the profile, the inferred sharding is approximately

$$
\begin{aligned}
B &\longmapsto X, \\
D &\longmapsto X, \\
H &\longmapsto Y, \\
V &\longmapsto Y, \\
F &\longmapsto Y.
\end{aligned}
$$

We also see an AllReduce over the Y axis at the end of each down projection, indicating that F is sharded over Y.

This is a nonstandard combination of tensor parallelism and FSDP. Normally, the up-projection weight has D sharded over Y, but here D is sharded over X.

Runtime Estimate

The dimensions inferred from the profile are

$$\begin{aligned} B &= 8, \\ D &= 256 \cdot 8 \cdot 4 = 8192, \\ F &= 16384 \cdot 2 = 32768. \end{aligned}$$

The profile shows that the MLP block takes approximately two thirds of the layer computation time.

The local token count is approximately

$1024 \cdot 2 = 2048$ tokens per chip.

Therefore, with FSDP and tensor parallelism, the MLP is compute bound.

The mathematical runtime of the MLP is estimated as

$$\begin{aligned} t_{\text{math,MLP}} &= \frac{2 \cdot 2 \cdot 8 \cdot 1024 \cdot 8192 \cdot 32768}{8\,(1.97 \times 10^{14})} \\ &\approx \boxed{5.58 \text{ ms}}. \end{aligned}$$

The attention operation is also compute bound because this is prefill with $T = 1024 > 240$. Its mathematical runtime is approximately

$$\begin{aligned} t_{\text{math,attention}} &= \frac{4 \cdot 8 \cdot 1024 \cdot 8192 \cdot 32 \cdot 256}{(1.97 \times 10^{14}) \cdot 8} \\ &\approx \boxed{1.4 \text{ ms}}. \end{aligned}$$

Therefore, the theoretical ratio is

$$\begin{aligned} \frac{t_{\text{math,MLP}}}{t_{\text{math,attention}}} &= \frac{5.6}{1.4} \\ &\approx 4. \end{aligned}$$

However, the empirical ratio observed in the profile is approximately 2. This indicates that the default profile is underutilizing the hardware.

Chapter 10

All About Jax

Q1

Let $A \in \texttt{float32}^{SX \times DY}$ be sharded over a mesh satisfying $XY = N$. For the experiment, I used a 4×2 mesh and an array of shape $A \in \mathbb{R}^{1024 \times 1024}$. The `shard_map` implementation computes the mean of each local shard directly: $\text{mean}\,(A_{\text{local}})$. The `jax.jit` implementation first reshapes the global array as $\left[X, \frac{S}{X}, Y, \frac{D}{Y}\right]$, permutes it to $\left[X, Y, \frac{S}{X}, \frac{D}{Y}\right]$, and then averages over the final two dimensions. The output therefore has shape $[X, Y]$. The outputs of the two implementations agree: $\text{jit\_avg}(A) = \text{shmap\_avg}(A)$. While profiling, I found that the `jax.jit` and `shard_map` implementations appeared to lower to the same XLA primitives. Calling one implementation could use the compiled code cached by the other. I verified this by switching their execution order in the profiler and observing that the displayed function name also switched.

Thus, XLA's automatic parallelization lowered the `jax.jit` implementation to the same computation as the explicit `shard_map` implementation. No collective communication was added.

I also implemented a roll along the local X-sharded dimension using `shard_map`. The result was checked against a reshape-based `jax.jit` reference. Since the roll occurs within each local shard, it does not require communication between devices.

The displayed notebook function computes the within-shard roll itself. The requested expression $\text{roll}(x, \textit{shift}, 0) - x$ is obtained by subtracting the original local array from the rolled array.

Q2

For the Mixture-of-Experts experiment, I used

$$
\begin{aligned}
E &= 8, \\
k &= 1, \\
D &= 2048, \\
F &= 8192, \\
S &= 1024.
\end{aligned}
$$

The activations, weights, and routes have shapes $X \in \mathbb{R}^{S \times D}$, $W \in \mathbb{R}^{E \times D \times F}$, and routes $\in \mathbb{Z}^{S \times k}$. The desired output is $Y_i = \sum_{j=1}^{k} X_i W_{\text{routes}_{i,j}}$.

Local and Automatically Sharded Implementation

The basic implementation loops over the experts. For expert e, it forms the mask $m_i = \mathbf{1}\left[\text{routes}_{i,0} = e\right]$, zeros the tokens not assigned to that expert, and computes $Y \mathrel{+}= (m \odot X)W_e$. A second reference implementation sorts the tokens by expert, computes the number of tokens assigned to each expert, applies `jax.lax.ragged_dot`, and then restores the original token order.

While profiling the basic `jax.jit` implementation, I observed that XLA AllGathered the full activation matrix onto each device. This is inefficient because each expert requires only the activations routed to that expert. It is also expensive in memory when the complete activation matrix cannot fit locally.

Explicit Ragged Communication

The explicit `shard_map` implementation performs the following steps:

1. Repeat each token k times and flatten the routing assignments.
2. Sort the repeated activations by expert.
3. Compute each expert's token count and offset.
4. Process the routed tokens in fixed-size chunks.
5. Use an `AllToAll` to send each expert's tokens to the device holding that expert.
6. Perform the local expert matrix multiplication.
7. Use a second `AllToAll` to return the outputs to their source devices.
8. Scatter the returned results into the sorted output buffer.
9. Undo the sorting, reshape to $[S, k, F]$, and sum over the k selected experts.

The maximum local padded expert size is combined with a `pmax`, ensuring that every device executes the same number of loop iterations. Processing the tokens inside a `jax.lax.while_loop` avoids materializing the full tensor $[E, S, D]$.

Timing Results

The measured runtimes were

$$t_{\text{naive}} = 19.7 \text{ ms} \pm 80.7 \ \mu\text{s},$$
$$t_{\text{explicit}} = 7.07 \text{ ms} \pm 99.7 \ \mu\text{s}.$$

The explicit sharded implementation is substantially faster because it avoids the full activation AllGather selected by XLA's automatic parallelization.

Q3

I implemented three collective matrix-multiplication experiments on a 2×4 mesh with axes X and Y.

AllReduce Collective Matrix Multiplication

The first operation has the form $A[B_X, D_Y] \ @_D \ W[D_Y, F] \longrightarrow O[B_X, F]$. The implementation tiles the output dimension F into one tile per device along Y. For each tile, it:

1. extracts the corresponding weight tile;
2. computes the local partial matrix multiplication;
3. applies `jax.lax.psum` over Y; and
4. writes the reduced result into the output buffer.

The measured runtimes were

$$t_{\text{collective}} = 1.13 \text{ ms} \pm 22.9 \ \mu\text{s},$$
$$t_{\text{jit}} = 1.11 \text{ ms} \pm 25.9 \ \mu\text{s}.$$

The custom implementation did not provide a speedup. This is consistent with the way XLA lowers and schedules the `psum`-based computation.

ReduceScatter Collective Matrix Multiplication

The complementary operation has the form $A[B_X, F_Y] @_F W[F_Y, D] \longrightarrow O[B_X, D_Y]$. Each device accumulates only the output tile needed by one destination device. During each iteration, it:

1. computes a local contribution to the current destination tile;
2. adds the contribution to a reduction buffer;

3. sends the buffer to the next device using `ppermute`; and
4. changes the target output tile for the next iteration.

After the ring completes, each device holds its fully reduced output tile. This avoids materializing or sending the complete output matrix.

The measured runtimes were

$$t_{\text{collective}} = 3.02 \text{ ms} \pm 84.5 \ \mu\text{s},$$
$$t_{\text{jit}} = 3.73 \text{ ms} \pm 71.3 \ \mu\text{s}.$$

The explicit ReduceScatter implementation provides a noticeable speedup over automatic parallelization.

End-to-End Transformer Block

The complete block computes $\text{In}[B_X, D_Y] @_D W_{\text{in}}[D, F_Y] @_F W_{\text{out}}[F_Y, D] \longrightarrow \text{Out}[B_X, D_Y]$. The up projection uses a collective AllGather matrix multiplication. The activation shard is circulated around the Y ring, and each device multiplies each received activation tile by the corresponding local weight tile.

The down projection uses the collective ReduceScatter matrix multiplication described above.

The measured runtimes were

$$t_{\text{collective}} = 1.13 \text{ ms} \pm 21.1 \ \mu\text{s},$$
$$t_{\text{jit}} = 1.18 \text{ ms} \pm 28.5 \ \mu\text{s}.$$

The collective implementation gives a small but measurable speedup.

Q4

The AllReduce implementation already uses `jax.lax.psum`, whose underlying collective communication is bidirectional. Therefore, I rewrote the ReduceScatter collective matrix multiplication to communicate in both directions.

The bidirectional implementation maintains two accumulation buffers, A_{left} and A_{right}. One buffer moves left around the Y ring, while the other moves right. Each iteration computes the local matrix-multiplication contribution for the target tile in each direction and then sends the corresponding accumulation buffer using `ppermute`.

After the left-moving and right-moving reductions complete, each device adds $A_{\text{local}} + A_{\text{left}} + A_{\text{right}}$ to produce its final output shard.

The measured runtimes were

$$t_{\text{bidirectional}} = 1.00 \text{ ms} \pm 26.1 \ \mu\text{s},$$
$$t_{\text{unidirectional}} = 1.02 \text{ ms} \pm 23.9 \ \mu\text{s}.$$

The difference is small because the Y axis contains only four devices. At this mesh size, the unidirectional path is already short, so bidirectional communication provides only a limited improvement.

Chapter 11: Conclusions

No exercises in this chapter.

Chapter 13

GPUs

Q13.1: GPU Hardware

Q1

For the H100, the number of FP32 cores is

$$\begin{aligned} N_{\text{H100}} &= 132 \text{ SMs} \times 4 \text{ subpartitions/SM} \times 32 \text{ cores/subpartition} \\ &= \boxed{16{,}896 \text{ cores}}. \end{aligned}$$

For the A100, the number of FP32 cores is

$$\begin{aligned} N_{\text{A100}} &= 108 \text{ SMs} \times 4 \text{ subpartitions/SM} \times 32 \text{ cores/subpartition} \\ &= \boxed{13{,}824 \text{ cores}}. \end{aligned}$$

For the B200, the number of FP32 cores is

$$\begin{aligned} N_{\text{B200}} &= 148 \text{ SMs} \times 4 \text{ subpartitions/SM} \times 32 \text{ cores/subpartition} \\ &= \boxed{18{,}944 \text{ cores}}. \end{aligned}$$

A TPUv5p has two cores. Using 8×128 ALUs per core and four units gives

$$\begin{aligned} N_{\text{TPUv5p}} &= 2(8)(128)(4) \\ &= \boxed{8192 \text{ ALUs}}. \end{aligned}$$

This is fewer than the corresponding GPU core count.

Q2

The number of cores is the number of ALUs. At the base clock, the H100 vector-operation throughput is

$$\begin{aligned} R_{\text{vector,base}} &= 16{,}896 \left(1.59 \times 10^9\right) \\ &= 2.69 \times 10^{13} \quad \text{vector operations/s.} \end{aligned}$$

At the boost clock,

$$\begin{aligned} R_{\text{vector,boost}} &= 16{,}896 \left(1.98 \times 10^9\right) \\ &= 3.35 \times 10^{13} \quad \text{vector operations/s.} \end{aligned}$$

The advertised Tensor Core throughput is approximately 9.89×10^{14} FLOPs/s. This value assumes structured sparsity, so the corresponding dense throughput is

$$\begin{aligned} R_{\text{tensor,dense}} &= \frac{9.89 \times 10^{14}}{2} \\ &= 4.95 \times 10^{14} \quad \text{FLOPs/s.} \end{aligned}$$

The ratio between matrix-multiplication FLOPs and vector-operation throughput is therefore

$$\frac{4.95 \times 10^{14}}{2.69 \times 10^{13}} \approx \boxed{18.4}.$$

Thus, the H100 provides approximately 18.4 times more dense matrix-multiplication FLOPs than vector operations.

Q3: Peak FP16 Matrix-Multiplication Intensity

The H100 has approximately 9.9×10^{14} FP16 FLOPs/s and 3.4×10^{12} bytes/s of HBM bandwidth.

Its FP16 accelerator intensity is therefore

$$\begin{aligned} I_{\text{H100,FP16}} &= \frac{9.9 \times 10^{14}}{3.4 \times 10^{12}} \\ &\approx \boxed{291}. \end{aligned}$$

For FP8, the compute throughput doubles while the HBM bandwidth remains unchanged:

$$\begin{aligned} I_{\text{H100,FP8}} &\approx 2(291) \\ &= \boxed{582}. \end{aligned}$$

For the B200,

$$\begin{aligned} I_{\text{B200,FP16}} &= \frac{2.3 \times 10^{15}}{8.0 \times 10^{12}} \\ &\approx \boxed{287}. \end{aligned}$$

For FP8,

$$\begin{aligned} I_{\text{B200,FP8}} &\approx 2(287) \\ &= \boxed{574}. \end{aligned}$$

Q4

Consider the FP16 matrix multiplication $[64, 4096] \text{ @ } [4096, 8192] \longrightarrow [64, 8192]$. For $B = 64$, the operation is communication bound. The communication time on the B200 is

$$\begin{aligned} t_{\text{comms}} &= \frac{2(64)(4096) + 2(4096)(8192) + 2(64)(8192)}{8.0 \times 10^{12}} \\ &\approx 0.008 \text{ ms} \\ &= \boxed{8\ \mu\text{s}}. \end{aligned}$$

Now consider $[512, 4096] \text{ @ } [4096, 8192] \longrightarrow [512, 8192]$. This operation is compute bound. Its mathematical runtime is

$$\begin{aligned} t_{\text{math}} &= \frac{2(512)(4096)(8192)}{2.3 \times 10^{15}} \\ &\approx \boxed{14.9\ \mu\text{s}}. \end{aligned}$$

Q5

The total H100 L1/shared-memory capacity is approximately

$$M_{\text{L1/SMEM}} = 132 \text{ SMs} \times 256 \text{ KiB}$$

$$\approx \boxed{33.7 \text{ MiB}}.$$

The total register capacity is approximately

$$\begin{aligned} M_{\text{registers}} &= 132 \text{ SMs} \times 256 \text{ KiB} \\ &\approx \boxed{33 \text{ MiB}}. \end{aligned}$$

A TPUv5e has approximately 128 MiB of VMEM and approximately 256 KiB of vector-register capacity per core.

Thus, the TPU has approximately four times as much VMEM as the H100 has L1/shared memory, but much less register capacity.

Q6

The B200 performs approximately 80×10^{12} FP32 vector FLOPs/s. It contains 18,944 FP32 cores. Each core performs two operations per cycle, so the number of operations per cycle is

$$18{,}944 \cdot 2 = 37{,}888.$$

The clock rate implied by the throughput is

$$\begin{aligned} f &= \frac{80 \times 10^{12}}{37{,}888} \\ &\approx 2.11 \times 10^{9} \quad \text{cycles/s}. \end{aligned}$$

Therefore, $\boxed{f \approx 2.11 \text{ GHz}}$.

Q7

The H100 provides approximately 2.69×10^{13} vector operations/s and 3.4×10^{12} bytes/s of HBM bandwidth.

Its vector-operation hardware intensity is therefore

$$I_{\text{hardware}} = \frac{2.69 \times 10^{13}}{3.4 \times 10^{12}}.$$

Consider adding two FP32 vectors of length N. The operation requires N floating-point operations and transfers three vectors: $4N + 4N + 4N = 12N$ bytes. Its arithmetic intensity is

$$\begin{aligned} I_{\text{vector add}} &= \frac{N}{12N} \\ &= \frac{1}{12}. \end{aligned}$$

The operation is therefore communication bound.

The compute and communication times are

$$\begin{aligned} t_{\text{math}} &= \frac{N}{2.69 \times 10^{13}}, \\ t_{\text{comms}} &= \frac{12N}{3.4 \times 10^{12}}. \end{aligned}$$

For $N = 1024$,

$$\begin{aligned} t_{\text{comms}} &= \frac{12(1024)}{3.4 \times 10^{12}} \\ &\approx \boxed{3.61 \text{ ns}}. \end{aligned}$$

For $N = 1024^3$,

$$\begin{aligned} t_{\text{comms}} &= \frac{12(1024^3)}{3.4 \times 10^{12}} \\ &\approx \boxed{3.78 \text{ ms}}. \end{aligned}$$

Q13.2: GPUs

Q1

Each NVLink has a full-duplex bandwidth of 25 GB/s. Each GPU has 18 links, so the egress bandwidth of one GPU is

$$\begin{aligned} W_{\text{GPU}} &= 18(25 \text{ GB/s}) \\ &= \boxed{450 \text{ GB/s}}. \end{aligned}$$

There are four switches, each containing 64 ports. Their aggregate switch bandwidth is

$$W_{\text{switches}} = 25 \text{ GB/s} \times 64 \times 4$$

$$= 6.4 \text{ TB/s}.$$

The eight GPUs can collectively inject

$$8(450 \text{ GB/s}) = 3.6 \text{ TB/s}.$$

The full-duplex bandwidth is therefore limited by the GPUs:

$$\boxed{W_{\text{full bisection}} = 3.6 \text{ TB/s}}.$$

Q2

Consider an arbitrary partition of the eight GPUs into two equal sets.

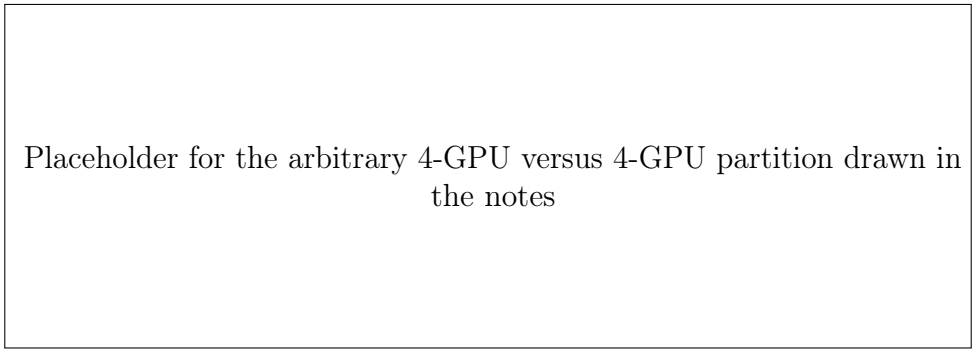

Figure 13.1: An arbitrary bisection of an eight-GPU NVLink domain.

Each GPU can egress 450 GB/s. Therefore, each half can inject

$$4(450 \text{ GB/s}) = 1.8 \text{ TB/s}.$$

The switches can simultaneously handle traffic in both directions:

$$1.8 \text{ TB/s} + 1.8 \text{ TB/s} = \boxed{3.6 \text{ TB/s}}.$$

Thus, the bisection bandwidth is 3.6 TB/s.

Q3

Consider an AllGather of an array containing B bytes over an eight-H100 NVLink domain.

Each GPU initially contains $\frac{B}{8}$ bytes. It must communicate its shard to the other seven GPUs, giving

$$t_{\text{AllGather}} = \frac{7}{8}\frac{B}{450\ \text{GB/s}}.$$

For a BF16 array $[D, F]$, with $D = 4096$, $F = 65536$, the volume is $B = 2DF$. Therefore,

$$\begin{aligned} t_{\text{AllGather}} &= \frac{(4096)(65536)(7)(2)}{8(450 \times 10^9)} \\ &\approx \boxed{1.04\ \text{ms}}. \end{aligned}$$

Q13.3: Scale-Out Networking

Q1

For a 1024-GPU system, the topology contains nodes, leaf switches, and spine switches.

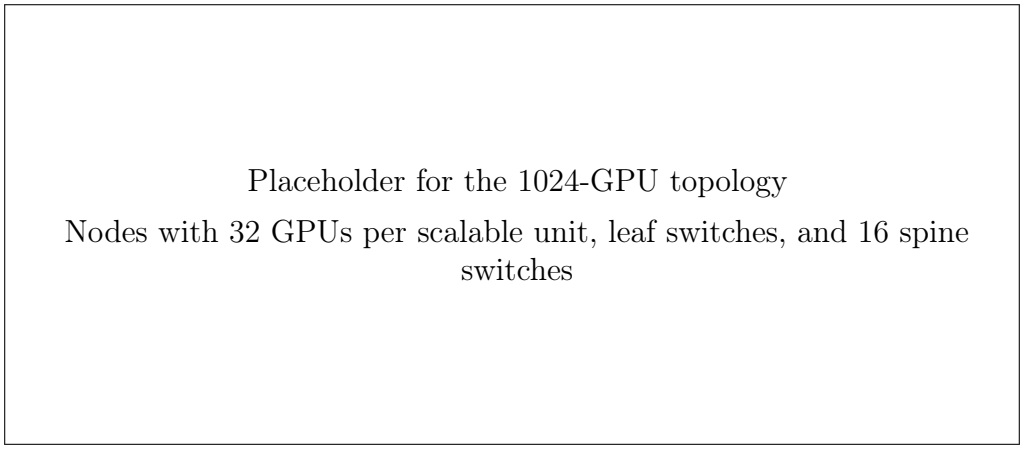

Figure 13.2: Leaf-and-spine topology for 1024 GPUs.

The aggregate traffic from the node layer to the leaf layer is

$$\begin{aligned} W_{\text{node}\to\text{leaf}} &= 50\ \text{GB/s} \times 8 \times 128 \\ &= \boxed{51.2\ \text{TB/s}}. \end{aligned}$$

The aggregate traffic between the leaf and spine layers is

$$\begin{aligned} W_{\text{leaf}\leftrightarrow\text{spine}} &= 32 \times 16 \times 100\ \text{GB/s} \\ &= \boxed{51.2\ \text{TB/s}}. \end{aligned}$$

Therefore, provided all links are fully utilized, the bisection bandwidth is $\boxed{51.2\ \text{TB/s}}$.

Q2

For 2048 GPUs, begin by duplicating the 1024-GPU structure.

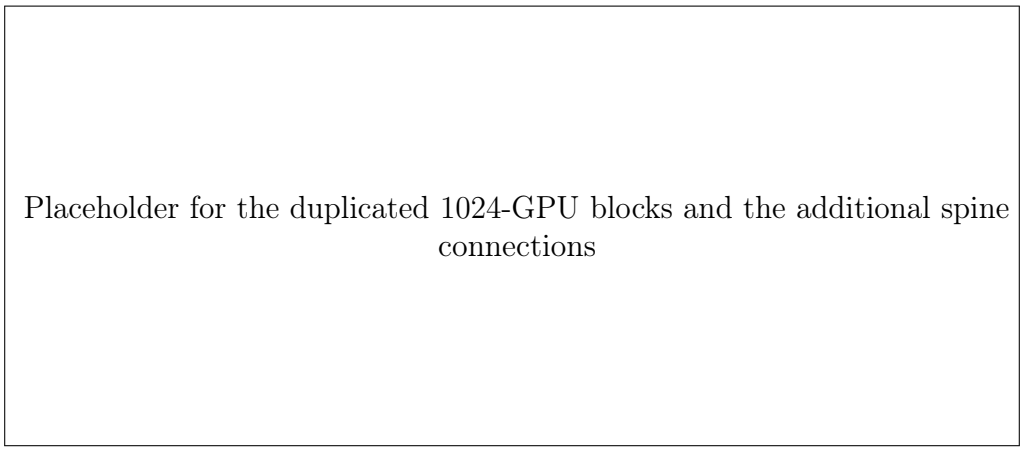

Figure 13.3: Extending the topology to 2048 GPUs.

Each block must be able to push approximately 51.2 TB/s into the spine layer.

This requires

$$64 \cdot 16 = 1024 \quad \text{ports},$$

and

$$1024(50 \text{ GB/s}) = 51.2 \text{ TB/s}.$$

Scaling to 2048 GPUs therefore requires 16 additional spine switches.

The two leaf layers jointly inject

$$2(51.2 \text{ TB/s}) = 102.4 \text{ TB/s}.$$

With 64 leaf switches, each leaf must send

$$\frac{102.4 \text{ TB/s}}{64} = 1.6 \text{ TB/s}$$

to the spine layer.

At 50 GB/s per port, this requires

$$\frac{1.6 \text{ TB/s}}{50 \text{ GB/s}} = 32$$

spine-facing ports per leaf switch.

Q3

For 4096 GPUs, there are 16 scalable units. The total traffic between the leaf and spine layers is

$$2(102.4 \text{ TB/s}) = \boxed{204.8 \text{ TB/s}}.$$

This requires approximately 4096 ports.

To obtain full bisection bandwidth, the 64 spine switches must connect to 128 leaf switches. However, each leaf can dedicate only 32 ports to spine communication.

The leaf switches cannot simply dedicate the same ports to an arbitrarily large number of spine switches. Regardless of the number of spines, the available paths out of each leaf remain limited.

The NVIDIA reference architecture recommends $\boxed{128 \text{ spine switches} + 64 \text{ core switches}}$. The precise routing through the additional core-switch layer is not fully resolved in the handwritten derivation.

Pop Quiz 1

For an AllGather over eight GPUs,

$$t_{\text{AllGather}} = \frac{B \cdot 7}{W_{\text{GPU,egress}} \cdot 8}.$$

For $B = 1024 \cdot 16384$ FP8 values,

$$\begin{aligned} t_{\text{AllGather}} &= \frac{(1024)(16384)(7)}{(450 \times 10^9)(8)} \\ &\approx \boxed{32\ \mu\text{s}}. \end{aligned}$$

For FP16, the volume doubles, giving $\boxed{64\ \mu\text{s}}$.

Pop Quiz 2

For an FP16 AllToAll over eight GPUs,

$$t_{\text{AllToAll}} = \frac{B(N-1)}{WN^2}.$$

Using $B = (1024)(16384)(2)$, $N = 8$, $W = 450 \times 10^9$, we obtain

$$\begin{aligned} t_{\text{AllToAll}} &= \frac{(1024)(16384)(7)(2)}{(450 \times 10^9)(64)} \\ &\approx \boxed{8.1\ \mu\text{s}}. \end{aligned}$$

If only four out of the eight routes are active, the corresponding time is approximately $\boxed{4.6\ \mu\text{s}}$.

Pop Quiz 3

Consider an AllGather over a scalable unit containing 32 nodes and eight GPUs per node.

The node-local communication time is

$$t_{\text{node}} = \frac{2DF}{450 \times 10^9} \frac{1}{\min(Y, 8)}.$$

The scale-out communication time is

$$t_{\text{scale-out}} = \frac{2DF}{400 \times 10^9} \frac{8}{\max(Y, 8)}.$$

The collective time is $t_{\text{comms}} = \max\left(t_{\text{node}}, t_{\text{scale-out}}\right)$. When $Y < 8$, the array is partially replicated within the node. Increasing Y improves the node-local communication, while the scale-out communication remains unchanged.

Q13.4: Hierarchical Collectives

Let $M =$ GPUs per node, $N =$ number of nodes. There are therefore MN GPUs in total, and the full array contains B bytes.

Q1: AllGather

Each GPU initially stores $\frac{B}{MN}$ bytes.

The local GPUs collectively send

$$M\left(\frac{B}{MN}\right) = \frac{B}{N}$$

bytes into the node-level switch.

The node also receives the shards from the other $N-1$ nodes:

$$\frac{B(N-1)}{N}.$$

Thus, the node-switch ingress is

$$\frac{B}{N} + \frac{B(N-1)}{N} = \boxed{B}.$$

Each of the M GPUs receives all data except its own local shard:

$$\frac{B(MN-1)}{MN}.$$

The total egress to the GPUs is therefore

$$M\frac{B(MN-1)}{MN} = \frac{B(MN-1)}{N}.$$

The node switch also sends $\frac{B}{N}$ bytes to the spine layer. Its total egress is therefore

$$\frac{B(MN-1)}{N} + \frac{B}{N} = \boxed{BM}.$$

The spine switch receives

$$N\left(\frac{B}{N}\right) = B$$

bytes in total.

It sends $\frac{B(N-1)}{N}$ bytes to each of the N nodes, for total egress

$$N\frac{B(N-1)}{N} = \boxed{B(N-1)}.$$

Q2: AllReduce

First consider an AllReduce within a node containing M GPUs.

Each GPU egresses everything except its locally retained shard:

$$\frac{B(M-1)}{M} \quad \text{bytes.}$$

Across all M GPUs, the node switch ingresses

$$M\frac{B(M-1)}{M} = B(M-1)$$

bytes.

The switch performs the reduction and sends a shard of size B/M to each GPU, for total egress B. This is the ReduceScatter phase.

The subsequent AllGather phase has switch ingress B and switch egress $B(M-1)$.

Thus, the total node-switch traffic is

$$\begin{aligned} \text{ingress} &= B(M-1) + B = \boxed{BM}, \\ \text{egress} &= B + B(M-1) = \boxed{BM}. \end{aligned}$$

Q3

For an AllReduce of a $2DF$-byte tensor over M GPUs in one node, the total switch traffic is proportional to $2DFM$. Because the node has M GPU links, the communication time is

$$t_{\text{AllReduce,node}} = \frac{2DF}{MW_{\text{egress}}}.$$

Multiple Nodes

If the reduction axis remains entirely within each node, the node-local reduction cost remains the same and the result is replicated across nodes.

If the reduction axis spans multiple nodes, each node can first perform a local reduction and the partially reduced results can then be combined across the scale-out network.

The communication cost depends on how the mesh axes X and Y are placed over the nodes.

If each independent reduction requires $\frac{2DF}{YW_{\text{egress,node}}}$ and there are X such reductions, the total cost is

$$t_{\text{comms}} = \frac{2DFX}{YW_{\text{egress,node}}}.$$

Q4

Using the bisection-bandwidth formula, the collective time is

$$t_{\text{comms}} = \frac{2DF}{W_{\text{bisection}}}.$$

For 256 GPUs, there are $\frac{256}{8} = 32$ nodes.

Using $W_{\text{bisection}} = 12.8$ TB/s, the time is

$$t_{\text{comms}} = \boxed{\frac{2DF}{12.8 \times 10^{12}}}.$$

Q5

Each GPU initially contains $\frac{B}{16}$ of the array.

At the node switch, the communication phases have aggregate volumes

1. ingress:
$$8\left(\frac{B}{16}\right) = \frac{B}{2},$$
2. egress:
$$\frac{B}{2},$$
3. ingress:
$$\frac{B}{2},$$
4. final egress:
$$8\left(\frac{15B}{16}\right) = \frac{15B}{2}.$$

The final egress dominates the node-local communication:

$$t_{\text{node}} = \frac{15B/2}{8(450 \times 10^9)}$$

$$= \frac{15B}{16(450 \times 10^9)}.$$

The inter-node communication is approximately

$$t_{\text{inter}} = \frac{B}{2(400 \times 10^9)}.$$

Comparing $\frac{B}{480 \times 10^9}$ and $\frac{B}{800 \times 10^9}$, the operation is bottlenecked by the intra-node communication.

Q13.5: Scaling Training

Q1

The accelerator compute throughput is approximately $C = 2.3 \times 10^{15}$ FLOPs/s. The intra-node collective bandwidth is approximately $W_{\text{intra}} = 900 \times 10^9$ bytes/s, so

$$\begin{aligned} \frac{C}{W_{\text{intra}}} &= \frac{2.3 \times 10^{15}}{900 \times 10^9} \\ &\approx \boxed{2556}. \end{aligned}$$

The inter-node collective bandwidth is approximately $W_{\text{inter}} = 400 \times 10^9$ bytes/s, so

$$\begin{aligned} \frac{C}{W_{\text{inter}}} &= \frac{2.3 \times 10^{15}}{400 \times 10^9} \\ &= \boxed{5750}. \end{aligned}$$

For tensor parallelism, the corresponding limits are

$$\begin{aligned} Y &< \frac{F}{2556} \qquad \text{within a node,} \\ Y &< \frac{F}{5750} \qquad \text{across nodes.} \end{aligned}$$

Q2

Memory

An H100 contains 80 GB of HBM.

A 70-billion-parameter model requires

$$\begin{aligned} M_{\text{parameters}} &= 70\text{B} \cdot 2 \\ &= 140 \text{ GB}. \end{aligned}$$

Including optimizer state gives approximately five times the parameter memory:

$$\begin{aligned} M_{\text{total}} &= 5(140 \text{ GB}) \\ &= 700 \text{ GB}. \end{aligned}$$

The number of H100 GPUs required is therefore

$$\frac{700}{80} = 8.75.$$

Thus, the model requires at least two eight-GPU nodes:

$$\boxed{16 \text{ H100 GPUs}}.$$

Training Time

The training FLOP count is

$$\begin{aligned} F_{\text{training}} &= 6 \times \text{tokens} \times \text{parameters} \\ &= 6 \left(15 \times 10^{12}\right) \left(70 \times 10^{9}\right) \\ &= 6.3 \times 10^{24} \quad \text{FLOPs}. \end{aligned}$$

Using 4096 H100 GPUs, approximately 9.9×10^{14} FLOPs/s per GPU, and 45% MFU,

$$\begin{aligned} t_{\text{training}} &= \frac{6.3 \times 10^{24}}{(4096)\,(9.9 \times 10^{14})\,(0.45)} \\ &\approx \boxed{40 \text{ days}}. \end{aligned}$$

Tensor Parallelism

For $F = 28672$, the first tensor-parallel limit is approximately

$$Y < \frac{28672}{2200}$$

$$\approx 13.$$

The second limit is approximately

$$\begin{aligned} Y &< \frac{28672}{2475} \\ &\approx 11.6. \end{aligned}$$

Therefore, an eight-way tensor-parallel configuration is used.

For the two-node case, the notes give the conditions

$$\begin{aligned} Y &< \frac{F}{2062} \approx 14, \\ Y &< \frac{F}{1237} \approx 23. \end{aligned}$$

Since the smaller limit is below 16, the selected configuration is $\boxed{\text{8-way tensor parallelism}}$. The handwritten conclusion labels this configuration as communication bound.

FSDP

With a global batch of $B = 4 \times 10^6$ tokens, pure 4096-way data parallelism gives

$$\begin{aligned} \frac{B}{X} &= \frac{4 \times 10^6}{4096} \\ &\approx 976. \end{aligned}$$

This is below the critical value 2062.

Using 1024-way partitioning gives

$$\begin{aligned} \frac{B}{X} &= \frac{4 \times 10^6}{1024} \\ &\approx 3906. \end{aligned}$$

The parameter and optimizer-state memory per GPU under full sharding is

$$\begin{aligned} M_{\text{per GPU}} &= \frac{70 \cdot 10}{4096} \\ &\approx 0.17 \text{ GB}. \end{aligned}$$

The notes select a combination of 1024-way FSDP and 8-way tensor parallelism, with a local batch of approximately $\frac{B}{X} \approx 4000$. This configuration is compute bound.

Pipeline Parallelism

With eight-way pipeline parallelism, the FSDP communication per stage falls by approximately a factor of eight.

The critical batch size therefore drops from 2062 to

$$\frac{2062}{8} \approx \boxed{258}.$$

The configuration remains compute bound.

Q3

For the 16-billion-parameter configuration, the batch size per replica is $BS = 192$. The global token batch is $B = 192 \cdot 4096$, and the per-GPU batch is written as

$$\begin{aligned} B_{\text{GPU}} &= \frac{192 \cdot 4096}{192} \\ &= \boxed{4096}. \end{aligned}$$

For the 70-billion-parameter configuration,

$$\begin{aligned} B_{\text{GPU}} &= \frac{384 \cdot 4096}{768} \\ &= \boxed{2048}. \end{aligned}$$

For the 31-billion-parameter configuration,

$$\begin{aligned} B_{\text{GPU}} &= \frac{156 \cdot 4096}{3072} \\ &\approx \boxed{208}. \end{aligned}$$

The first configuration is immediately compute bound with FSDP.

For the 70-billion-parameter configuration, $B_{\text{GPU}} = 2048$, which is slightly below the intra-node threshold of approximately 2475.

Using two-way pipeline parallelism reduces the FSDP critical batch size by a factor of two:

$$B_{\text{crit}} \approx \frac{2475}{2}$$

$$\approx \boxed{1230}.$$

For the smaller configuration, the communication can be approximately eight times faster, giving a critical batch size near $\boxed{300}$. The notes conclude that these configurations are already sufficiently compute bound and that substantially different parallelism configurations are unlikely to provide a large improvement.

Appendix A

Chapter 10 Code Implementations

Question 1: Local Shard Operations

```python
import functools
import numpy as np

import jax
import jax.numpy as jnp
from jax.sharding import Mesh, PartitionSpec as P

Explicit = jax.sharding.AxisType.Explicit
Auto = jax.sharding.AxisType.Auto


mesh = jax.make_mesh(
    axis_shapes=(4, 2),
    axis_names=("X", "Y"),
    axis_types=(Auto, Auto),
)
jax.set_mesh(mesh)

key = jax.random.key(0)

A = jax.random.normal(key, (1024, 1024))
A = jax.device_put(A, P("X", "Y"))


@jax.jit
@jax.shard_map(
    in_specs=P("X", "Y"),
    out_specs=P("X", "Y"),
)
def shmap_avg(A_matrix):
    return jnp.mean(A_matrix, keepdims=True)


@jax.jit
def jit_avg(A_matrix):
    X_axis = mesh.shape["X"]
    Y_axis = mesh.shape["Y"]

```

```
    A_matrix = jnp.reshape(
        A_matrix,
        (
            X_axis,
            A.shape[0] // X_axis,
            Y_axis,
            A.shape[1] // Y_axis,
        ),
    )

    A_matrix = jnp.permute_dims(
        A_matrix,
        (0, 2, 1, 3),
    )

    return jnp.mean(
        A_matrix,
        axis=(-2, -1),
    )


assert jnp.allclose(
    jit_avg(A),
    shmap_avg(A),
)


def s_roll(x, shift):
    @jax.shard_map(
        in_specs=P("X", "Y"),
        out_specs=P("X", "Y"),
    )
    def shmap_roll(x):
        return jnp.roll(
            x,
            shift,
            axis=0,
        )

    return shmap_roll(x)


@functools.partial(
    jax.jit,
    static_argnames=["shift"],
    out_shardings=jax.NamedSharding(
        mesh,
        P("X", "Y"),
    ),
)
def shift_jit(x, shift: int):
    X, Y = mesh.axis_sizes

    reshaped = x.reshape(
        X,
        x.shape[0] // X,
        -1,
    )
```

```
    return jnp.roll(
        reshaped,
        shift,
        axis=1,
    ).reshape(
        x.shape[0],
        x.shape[1],
    )


x = jnp.arange(
    8 * 64 * 8,
    dtype=jnp.float32,
).reshape(
    8 * 64,
    8,
)

x = jax.device_put(
    x,
    jax.NamedSharding(
        mesh,
        P("X", "Y"),
    ),
)

y1 = s_roll(x, 5)
y2 = shift_jit(x, 5)

np.testing.assert_array_equal(y1, y2)


# Profiling
assert jnp.allclose(
    shmap_avg(A),
    jit_avg(A),
)

with jax.profiler.trace("/kaggle/working/"):
    shmap_avg(A).block_until_ready()
    jit_avg(A).block_until_ready()
```

Question 2: Mixture of Experts

```
mesh = jax.make_mesh(
    axis_shapes=(8,),
    axis_names="X",
    axis_types=(Auto),
)
jax.set_mesh(mesh)

keys = jax.random.split(
    jax.random.key(2),
    6,
```

```
)

E = 8
k = 1
D = 2048
F = 8192
S = 1024

num_devices = 8
BLOCK_SIZE = 512


X = jax.device_put(
    jax.random.normal(
        keys[0],
        (S, D),
        dtype=jnp.float32,
    ),
    P("X", None),
)

W = jax.device_put(
    jax.random.normal(
        keys[1],
        (E, D, F),
        dtype=jnp.float32,
    ),
    P("X", None, None),
)

routes = jax.device_put(
    jax.random.categorical(
        keys[3],
        jnp.zeros((E,)),
        axis=0,
        shape=(S, k),
    ),
    P("X"),
)


@jax.jit
def moe_basic(X, W, routes):
    out = jnp.zeros((S, F))

    for i in range(E):
        mask = routes[:, 0] == i

        masked = jnp.where(
            mask[:, None],
            X,
            0,
        ).astype(float)

        out += masked @ W[i]

    return out

```

```
69
70 @jax.jit
71 def moe_truth(X, W, routes):
72     packed_X = jnp.repeat(
73         X,
74         k,
75         axis=0,
76     )
77
78     flattened_routes = routes.flatten()
79
80     g = jnp.bincount(
81         flattened_routes,
82         length=E,
83     )
84
85     args = jnp.argsort(flattened_routes)
86     unsort_args = jnp.argsort(args)
87
88     sorted_X = packed_X[args]
89
90     result = jax.lax.ragged_dot(
91         sorted_X,
92         W,
93         g,
94     )
95
96     Y = result[unsort_args]
97     Y = Y.reshape(S, k, F)
98     Y = Y.sum(axis=1)
99
100    return Y
101
102
103 @jax.jit
104 @jax.shard_map(
105     mesh=mesh,
106     in_specs=P("X"),
107     out_specs=P("X"),
108 )
109 def moe_shard(X, W, routes):
110     packed_X = jnp.repeat(
111         X,
112         k,
113         axis=0,
114     )
115
116     flattened_routes = routes.flatten()
117
118     args = jnp.argsort(flattened_routes)
119     unsort_args = jnp.argsort(args)
120
121     sorted_X = packed_X[args]
122
123     # The code below this point does not depend on k.
124     g = jnp.bincount(
125         flattened_routes,
126         length=E,
```

```
    )

    g_sum = jnp.cumsum(g)

    g_offset = jnp.cumsum(
        jnp.concatenate(
            (
                jnp.zeros(1),
                g[:-1],
            )
        )
    ).astype(int)

    blocks = jnp.ceil(
        jnp.max(g) / BLOCK_SIZE
    ).astype(int)

    local_padded_expert_size = jnp.array(
        BLOCK_SIZE * blocks
    )

    # Find the global maximum so all devices pad evenly.
    global_padded_expert_size = jax.lax.pmax(
        local_padded_expert_size,
        "X",
    )

    def cond(carry):
        i, Y_sorted = carry
        return i < global_padded_expert_size

    def body(carry):
        i, Y_sorted = carry

        chunk_idx = i + jnp.arange(BLOCK_SIZE)

        offset_indices = (
            g_offset[:, None]
            +
            chunk_idx[None, :]
        )

        offset_mask = (
            chunk_idx[None, :]
            <
            g[:, None]
        )

        indices = jnp.where(
            offset_mask,
            offset_indices,
            0,
        )

        X_e = jnp.where(
            offset_mask[:, :, None],
            sorted_X[indices],
            0,
```

```
        )

        X_commed = jax.lax.all_to_all(
            X_e,
            "X",
            split_axis=0,
            concat_axis=0,
            tiled=False,
        )

        # Local expert matrix multiplication.
        result = X_commed @ W[0]

        # Return results to the source devices.
        activated = jax.lax.all_to_all(
            result,
            "X",
            split_axis=0,
            concat_axis=0,
            tiled=False,
        )

        # Retrieve and unpack the returned results.
        flattened_mask = offset_mask.flatten()[:, None]
        flattened_indices = offset_indices.flatten()[:, None]

        unpacked_mask = jnp.where(
            flattened_mask,
            flattened_indices,
            0,
        ).reshape(E * BLOCK_SIZE)

        unpacked_result = activated.reshape(
            E * BLOCK_SIZE,
            F,
        )

        unpacked_result = jnp.where(
            flattened_mask,
            unpacked_result,
            0,
        )

        Y_sorted = Y_sorted.at[
            unpacked_mask
        ].add(
            unpacked_result
        )

        return (
            i + BLOCK_SIZE,
            Y_sorted,
        )

    # This is a globally sharded output buffer.
    Y_sorted_init = jax.device_put(
        jnp.zeros((S * k, F)),
        P("X"),
```

```
    )

    # Each device accumulates independently.
    Y_sorted_init = jax.lax.pcast(
        Y_sorted_init,
        "X",
        to="varying",
    )

    init_i = jnp.array(0)

    _, Y_sorted = jax.lax.while_loop(
        cond,
        body,
        (
            init_i,
            Y_sorted_init,
        ),
    )

    Y = Y_sorted[unsort_args]

    Y = Y.reshape(
        S // num_devices,
        k,
        F,
    )

    Y = Y.sum(axis=1)

    return Y


difference = jnp.abs(
    moe_basic(X, W, routes)
    -
    moe_shard(X, W, routes)
)

print(jnp.max(difference))


# Profile the automatically sharded implementation.
moe_basic(X, W, routes).block_until_ready()

with jax.profiler.trace("/kaggle/working/"):
    moe_basic(
        X,
        W,
        routes,
    ).block_until_ready()


# Timing
%timeit -n 10 -r 100 \
    moe_basic(X, W, routes).block_until_ready()

%timeit -n 10 -r 100 \
```

```
    moe_shard(X, W, routes).block_until_ready()
```

Question 3.1: AllReduce Collective Matmul

```
Explicit = jax.sharding.AxisType.Explicit

mesh = jax.make_mesh(
    axis_shapes=(2, 4),
    axis_names=("X", "Y"),
    axis_types=(Explicit, Explicit),
)
jax.set_mesh(mesh)

B, D, F = 1024, 2048, 8192

A = jnp.arange(
    np.prod((B, D))
).reshape(
    (B, D)
)

W = jnp.arange(
    np.prod((D, F))
).reshape(
    (D, F)
)

A = jax.device_put(
    A,
    P("X", "Y"),
)

W = jax.device_put(
    W,
    P("Y", None),
)


@functools.partial(
    jax.jit,
    out_shardings=P("X", None),
)
def matmul(lhs, rhs):
    return jax.lax.dot(
        lhs,
        rhs,
        out_sharding=P("X", None),
    )


def collective_matmul_all_reduce(lhs, rhs):
    # Tile over F.
    axis_size = jax.lax.axis_size("Y")
    idx = jax.lax.axis_index("Y")

```

```
    assert rhs.shape[1] % axis_size == 0

    tile_size = rhs.shape[1] // axis_size

    def work(i, carry):
        accum, lhs = carry

        rhs_chunk = jax.lax.dynamic_slice_in_dim(
            rhs,
            i * tile_size,
            tile_size,
            axis=1,
        )

        # Compute one local output tile.
        update = lhs @ rhs_chunk

        # AllReduce the partial result.
        all_reduced = jax.lax.psum(
            update,
            "Y",
        )

        accum = jax.lax.dynamic_update_slice(
            accum,
            all_reduced,
            (
                0,
                i * tile_size,
            ),
        )

        return accum, lhs

    accum = jnp.zeros(
        (
            lhs.shape[0],
            rhs.shape[1],
        ),
        dtype=lhs.dtype,
    )

    accum = jax.lax.pcast(
        accum,
        ("X",),
        to="varying",
    )

    accum, lhs = jax.lax.fori_loop(
        0,
        axis_size,
        work,
        (
            accum,
            lhs,
        ),
        unroll=True,
    )
```

```
    return accum


jit_sharded_all_reduce = jax.jit(
    jax.shard_map(
        collective_matmul_all_reduce,
        in_specs=(
            P("X", "Y"),
            P("Y", None),
        ),
        out_specs=P("X", None),
        check_vma=True,
    )
)


shmapped_out = jit_sharded_all_reduce(
    A,
    W,
)

expected_out = matmul(
    A,
    W,
)

np.testing.assert_array_equal(
    shmapped_out,
    expected_out,
)


%timeit -r 100 -n 10 \
    jit_sharded_all_reduce(A, W).block_until_ready()

%timeit -r 100 -n 10 \
    matmul(A, W).block_until_ready()
```

Question 3.2: ReduceScatter Collective Matmul

```
Explicit = jax.sharding.AxisType.Explicit

mesh = jax.make_mesh(
    axis_shapes=(2, 4),
    axis_names=("X", "Y"),
    axis_types=(Explicit, Explicit),
)
jax.set_mesh(mesh)

B, D, F = 2048, 4096, 16384

A = jnp.arange(
    np.prod((B, D))
).reshape(
```

```
    (B, D)
)

W = jnp.arange(
    np.prod((D, F))
).reshape(
    (D, F)
)

A = jax.device_put(
    A,
    P("X", "Y"),
)

W = jax.device_put(
    W,
    P("Y", None),
)


@functools.partial(
    jax.jit,
    out_shardings=P("X", None),
)
def matmul(lhs, rhs):
    return jax.lax.dot(
        lhs,
        rhs,
        out_sharding=P("X", None),
    )


def collective_matmul_reduce_scatter(lhs, rhs):
    axis_size = jax.lax.axis_size("Y")
    idx = jax.lax.axis_index("Y")

    chunk_size = rhs.shape[1] // axis_size

    perms = [
        (
            j,
            (j + 1) % axis_size,
        )
        for j in range(axis_size)
    ]

    def local_matmul(target_idx):
        rhs_chunk = jax.lax.dynamic_slice_in_dim(
            rhs,
            target_idx * chunk_size,
            chunk_size,
            axis=1,
        )

        return lhs @ rhs_chunk

    def work(i, carry):
        target_idx, accum = carry
```

```
73
74          # Add this device's contribution to the target tile.
75          accum += local_matmul(target_idx)
76
77          # Send the partial reduction to the right.
78          accum = jax.lax.ppermute(
79              accum,
80              "Y",
81              perms,
82          )
83
84          # The next target is one device to the left.
85          target_idx = (
86              target_idx - 1
87          ) % axis_size
88
89          return target_idx, accum
90
91      accum = jnp.zeros(
92          (
93              lhs.shape[0],
94              rhs.shape[1] // axis_size,
95          ),
96          dtype=lhs.dtype,
97      )
98
99      accum = jax.lax.pcast(
100         accum,
101         ("X", "Y"),
102         to="varying",
103     )
104
105     # Begin with the target immediately to the left.
106     target_idx = (
107         idx - 1
108     ) % axis_size
109
110     _, accum = jax.lax.fori_loop(
111         0,
112         axis_size - 1,
113         work,
114         (
115             target_idx,
116             accum,
117         ),
118         unroll=True,
119     )
120
121     # Add the contribution to the locally owned tile.
122     accum += local_matmul(idx)
123
124     return accum
125
126
127 jit_sharded_cmrs = jax.jit(
128     jax.shard_map(
129         collective_matmul_reduce_scatter,
130         in_specs=(
```

```
            P("X", "Y"),
            P("Y", None),
        ),
        out_specs=P("X", "Y"),
        check_vma=True,
    )
)


shmapped_out = jit_sharded_cmrs(
    A,
    W,
)

expected_out = matmul(
    A,
    W,
)

np.testing.assert_array_equal(
    shmapped_out,
    expected_out,
)


%timeit -r 100 -n 10 \
    jit_sharded_cmrs(A, W).block_until_ready()

%timeit -r 100 -n 10 \
    matmul(A, W).block_until_ready()
```

Question 3.3: End-to-End Transformer Block

```
Explicit = jax.sharding.AxisType.Explicit

mesh = jax.make_mesh(
    axis_shapes=(2, 4),
    axis_names=("X", "Y"),
    axis_types=(Explicit, Explicit),
)
jax.set_mesh(mesh)

B, D, F = 2048, 4096, 16384

A = jnp.arange(
    np.prod((B, D))
).reshape(
    (B, D)
)

W1 = jnp.arange(
    np.prod((D, F))
).reshape(
    (D, F)
)
```

```
23
24 W2 = jnp.arange(
25     np.prod((D, F))
26 ).reshape(
27     (F, D)
28 )
29
30 A = jax.device_put(
31     A,
32     P("X", "Y"),
33 )
34
35 W1 = jax.device_put(
36     W1,
37     P(None, "Y"),
38 )
39
40 W2 = jax.device_put(
41     W2,
42     P("Y", None),
43 )
44
45
46 @functools.partial(
47     jax.jit,
48     out_shardings=P("X", "Y"),
49 )
50 def transformer_block_ref(A, W1, W2):
51     up_proj = A @ W1
52
53     down_proj = jax.lax.dot(
54         up_proj,
55         W2,
56         out_sharding=P("X", "Y"),
57     )
58
59     return down_proj
60
61
62 def collective_matmul_allgather(lhs, rhs):
63     axis_size = jax.lax.axis_size("Y")
64     idx = jax.lax.axis_index("Y")
65
66     chunk_size = lhs.shape[1]
67
68     assert rhs.shape[0] % chunk_size == 0
69
70     permutations = [
71         (
72             j,
73             (j - 1) % axis_size,
74         )
75         for j in range(axis_size)
76     ]
77
78     def work(i, carry):
79         accum, lhs = carry
80
```

```
        rhs_chunk = jax.lax.dynamic_slice_in_dim(
            rhs,
            (
                (idx + i) % axis_size
            ) * chunk_size,
            chunk_size,
            axis=0,
        )

        update = lhs @ rhs_chunk

        # Circularly shift the activation tile to the left.
        lhs = jax.lax.ppermute(
            lhs,
            axis_name="Y",
            perm=permutations,
        )

        return accum + update, lhs

    accum = jnp.zeros(
        (
            lhs.shape[0],
            rhs.shape[1],
        ),
        dtype=lhs.dtype,
    )

    accum = jax.lax.pcast(
        accum,
        ("X", "Y"),
        to="varying",
    )

    accum, lhs = jax.lax.fori_loop(
        0,
        axis_size - 1,
        work,
        (
            accum,
            lhs,
        ),
        unroll=True,
    )

    # Compute the final tile after the final permutation.
    i = axis_size - 1

    rhs_chunk = jax.lax.dynamic_slice_in_dim(
        rhs,
        (
            (idx + i) % axis_size
        ) * chunk_size,
        chunk_size,
        axis=0,
    )

    update = lhs @ rhs_chunk
```

```
    return accum + update


def collective_transformer_block(
    A,
    W1,
    W2,
):
    up_proj = collective_matmul_allgather(
        A,
        W1,
    )

    down_proj = collective_matmul_reduce_scatter(
        up_proj,
        W2,
    )

    return down_proj


jit_sharded_block = jax.jit(
    jax.shard_map(
        collective_transformer_block,
        in_specs=(
            P("X", "Y"),
            P(None, "Y"),
            P("Y", None),
        ),
        out_specs=P("X", "Y"),
    )
)


shmapped_out = jit_sharded_block(
    A,
    W1,
    W2,
)

expected_out = transformer_block_ref(
    A,
    W1,
    W2,
)

np.testing.assert_array_equal(
    shmapped_out,
    expected_out,
)


%timeit -n 10 -r 100 \
    jit_sharded_block(A, W1, W2).block_until_ready()

%timeit -n 10 -r 100 \
    transformer_block_ref(A, W1, W2).block_until_ready()
```

Question 4: Bidirectional ReduceScatter

```
Explicit = jax.sharding.AxisType.Explicit

mesh = jax.make_mesh(
    axis_shapes=(2, 4),
    axis_names=("X", "Y"),
    axis_types=(Explicit, Explicit),
)
jax.set_mesh(mesh)

B, D, F = 1024, 2048, 8192

A = jnp.arange(
    np.prod((B, D))
).reshape(
    (B, D)
)

W = jnp.arange(
    np.prod((D, F))
).reshape(
    (D, F)
)

A = jax.device_put(
    A,
    P("X", "Y"),
)

W = jax.device_put(
    W,
    P("Y", None),
)


@functools.partial(
    jax.jit,
    out_shardings=P("X", None),
)
def matmul(lhs, rhs):
    return jax.lax.dot(
        lhs,
        rhs,
        out_sharding=P("X", None),
    )


def bidirectional_collective_matmul_reduce_scatter(
    lhs,
    rhs,
):
    axis_size = jax.lax.axis_size("Y")
    idx = jax.lax.axis_index("Y")

    chunk_size = rhs.shape[1] // axis_size

    right_perms = [
```

```
        (
            j,
            (j + 1) % axis_size,
        )
        for j in range(axis_size)
    ]

    left_perms = [
        (
            j,
            (j - 1) % axis_size,
        )
        for j in range(axis_size)
    ]

    def local_matmul(target_idx):
        rhs_chunk = jax.lax.dynamic_slice_in_dim(
            rhs,
            target_idx * chunk_size,
            chunk_size,
            axis=1,
        )

        return lhs @ rhs_chunk

    def work(i, carry):
        (
            left_target_idx,
            right_target_idx,
            left_accum,
            right_accum,
        ) = carry

        left_accum += local_matmul(
            left_target_idx
        )

        left_accum = jax.lax.ppermute(
            left_accum,
            "Y",
            left_perms,
        )

        right_accum += local_matmul(
            right_target_idx
        )

        right_accum = jax.lax.ppermute(
            right_accum,
            "Y",
            right_perms,
        )

        right_target_idx = (
            right_target_idx - 1
        ) % axis_size

        left_target_idx = (
```

```
            left_target_idx + 1
        ) % axis_size

        return (
            left_target_idx,
            right_target_idx,
            left_accum,
            right_accum,
        )

    left_accum = jnp.zeros(
        (
            lhs.shape[0],
            rhs.shape[1] // axis_size,
        ),
        dtype=lhs.dtype,
    )

    left_accum = jax.lax.pcast(
        left_accum,
        ("X", "Y"),
        to="varying",
    )

    right_accum = jnp.zeros(
        (
            lhs.shape[0],
            rhs.shape[1] // axis_size,
        ),
        dtype=lhs.dtype,
    )

    right_accum = jax.lax.pcast(
        right_accum,
        ("X", "Y"),
        to="varying",
    )

    # Begin with the destination furthest from this source.
    right_target_idx = (
        idx + axis_size // 2
    ) % axis_size

    left_target_idx = (
        idx + axis_size // 2
    ) % axis_size

    # The first iteration sends only to the right.
    right_accum += local_matmul(
        right_target_idx
    )

    right_accum = jax.lax.ppermute(
        right_accum,
        "Y",
        right_perms,
    )

```

```
    right_target_idx = (
        right_target_idx - 1
    ) % axis_size

    left_target_idx = (
        left_target_idx + 1
    ) % axis_size

    (
        _,
        _,
        left_accum,
        right_accum,
    ) = jax.lax.fori_loop(
        0,
        axis_size // 2 - 1,
        work,
        (
            left_target_idx,
            right_target_idx,
            left_accum,
            right_accum,
        ),
        unroll=True,
    )

    # Add the contribution to the locally owned tile.
    accum = (
        local_matmul(idx)
        +
        left_accum
        +
        right_accum
    )

    return accum


jit_sharded_bidirectional = jax.jit(
    jax.shard_map(
        bidirectional_collective_matmul_reduce_scatter,
        in_specs=(
            P("X", "Y"),
            P("Y", None),
        ),
        out_specs=P("X", "Y"),
        check_vma=True,
    )
)


shmapped_out = jit_sharded_bidirectional(
    A,
    W,
)

expected_out = matmul(
    A,
```

```
    W,
)

np.testing.assert_array_equal(
    shmapped_out,
    expected_out,
)


%timeit -r 100 -n 10 \
    jit_sharded_bidirectional(A, W).block_until_ready()

%timeit -r 100 -n 10 \
    jit_sharded_cmrs(A, W).block_until_ready()
```